



上海科技大学
ShanghaiTech University

Lecture 03: RNNs

Zhuyu Lin
Spring, 2026

Outline



■ Introduction to RNNs

- Formula and Architecture
- Sequential Tasks with RNNs
- Problems with RNNs

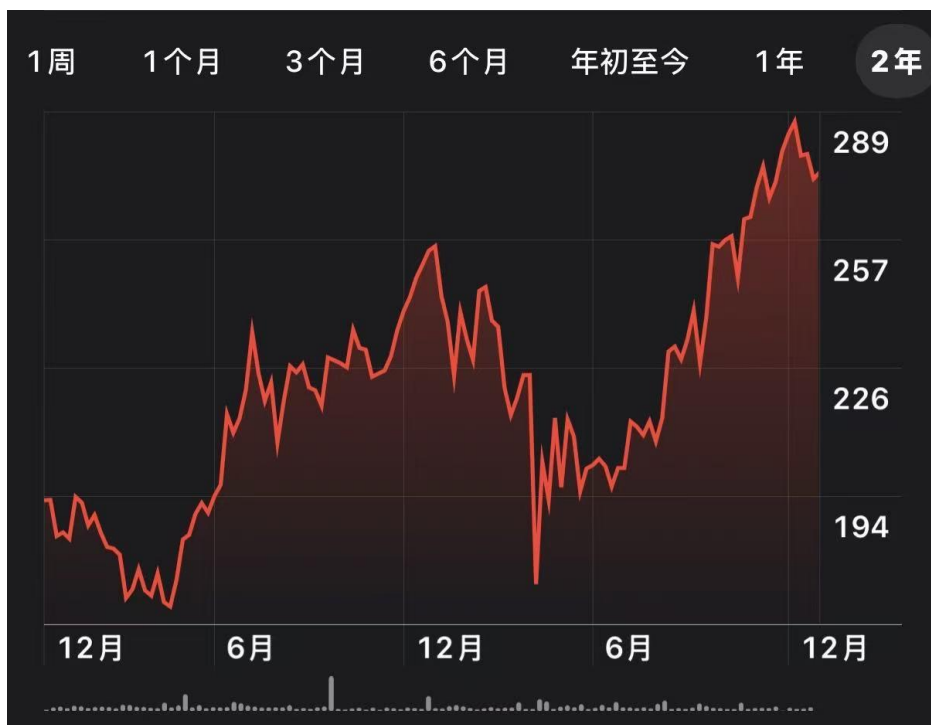
■ Long Short-Term Memory (LSTM)

■ Seq2Seq Model and Attention

sequential data



- Order matters: the order of elements influences the meaning
- For example: text data, stock price, speech data, DNA sequence



我爱你 $\neq$ 你爱我

我做了一个_____

蛋糕

梦

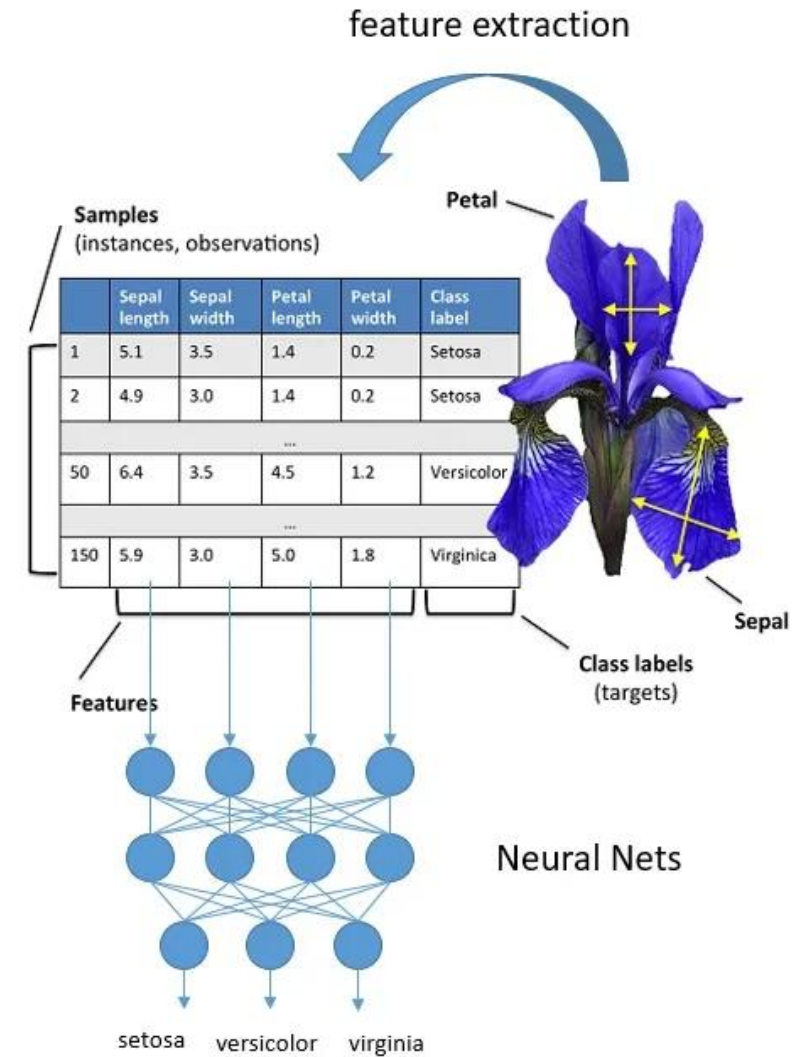
测试

决定

non sequential data



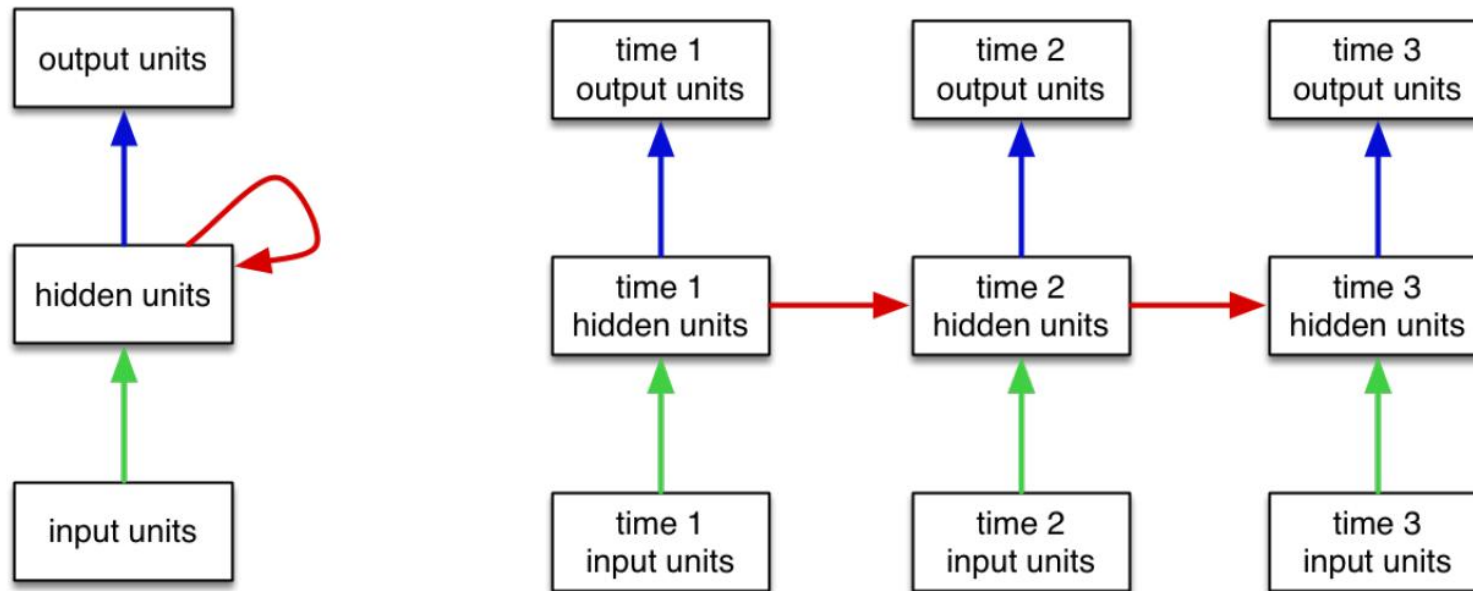
- Order doesn't matter
- For example: tabular data, image classification



Recurrent Neural Network



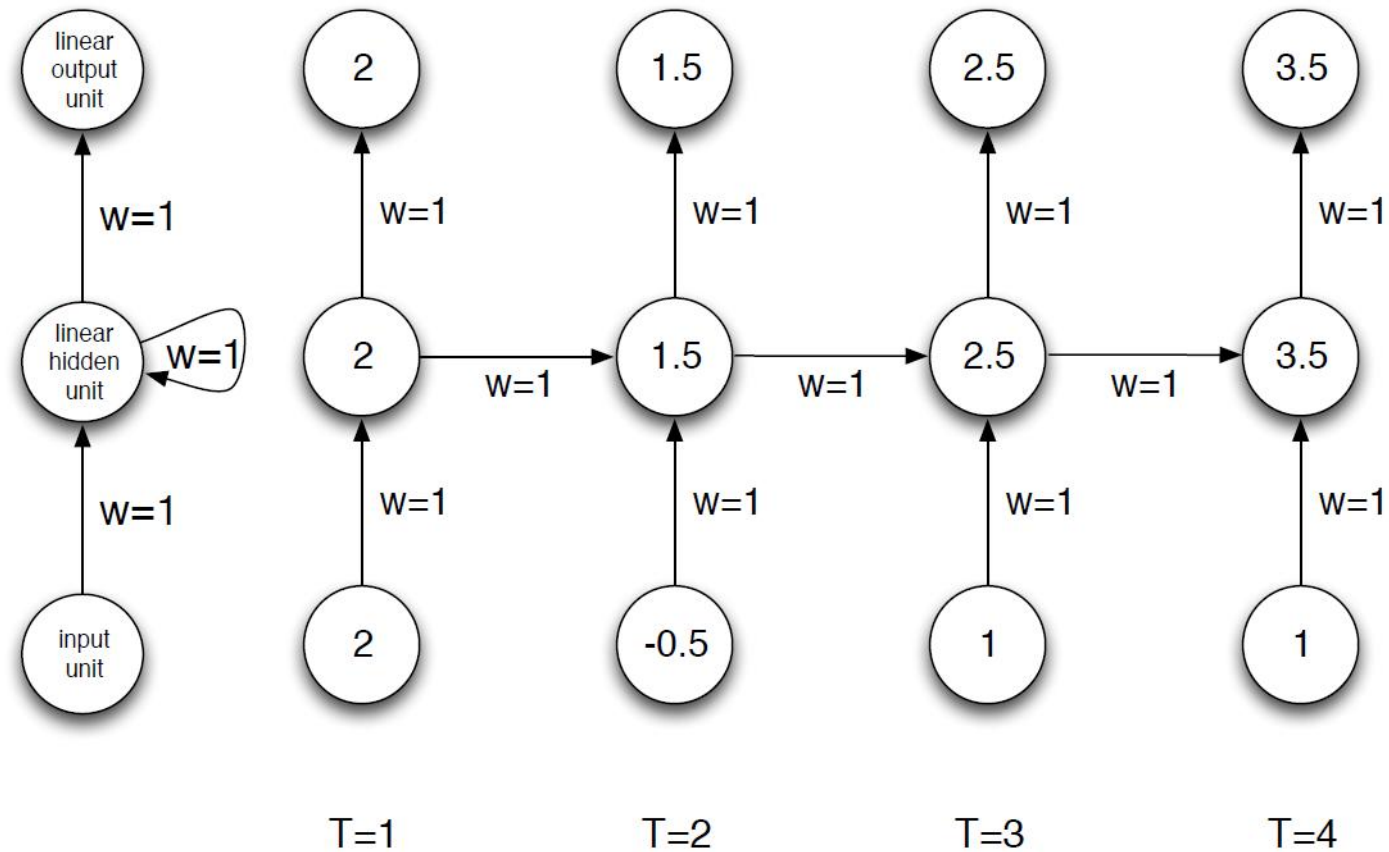
- Recurrent Neural Network as a dynamical system with one set of hidden units feeding into themselves
 - The network's graph has self-loops
- The RNN's graph can be unrolled by explicitly representing the units at all time steps
 - The weights and biases are shared



RNN examples



■ Summation network



Recurrent Neural Network



■ General formula: RNN hidden state update

- We can process a sequence of vectors x by applying a recurrence formula at every time step:

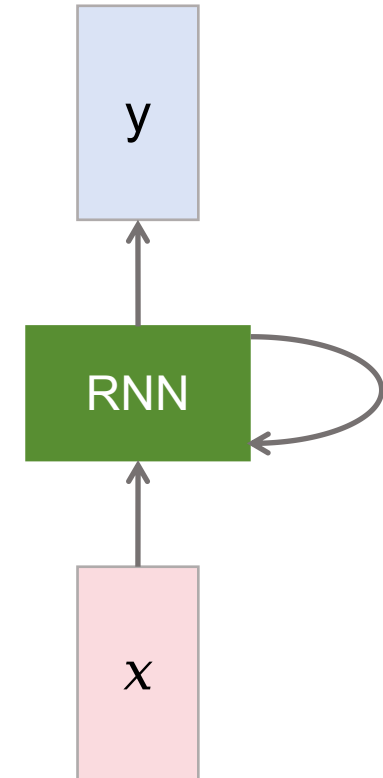
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

some function with parameters W

old state

input vector at some time step



Recurrent Neural Network



■ General formula: RNN hidden state update

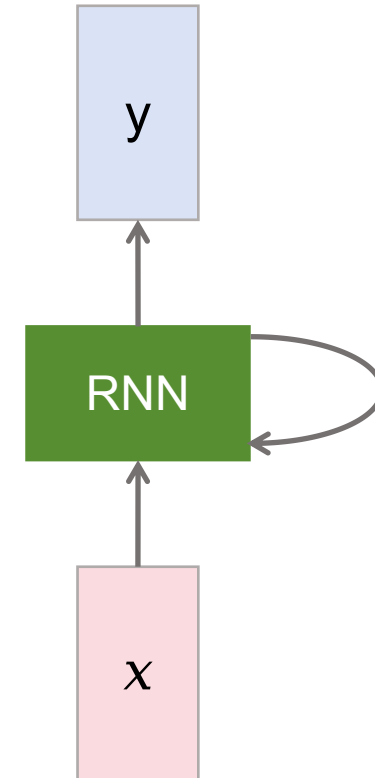
- We can process a sequence of vectors x by applying a recurrence formula at every time step:

$$\boxed{y_t} = \boxed{g_{W_{hy}}}(h_t)$$

output

another function with parameters W_{hy}

new state



Recurrent Neural Network



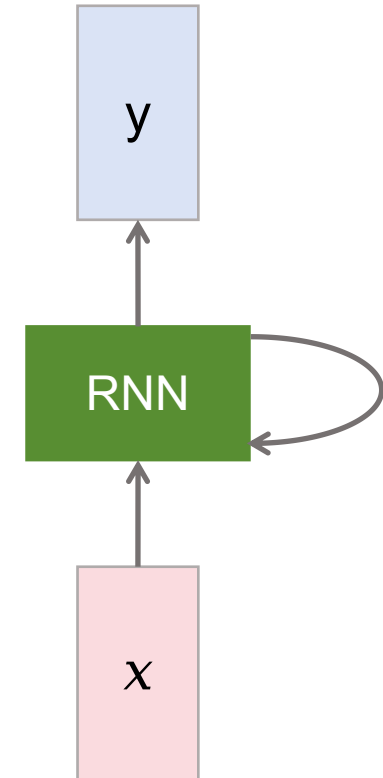
■ General formula: RNN hidden state update

- We can process a sequence of vectors x by applying a recurrence formula at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

$$y_t = g_{W_{hy}}(h_t)$$

Notice: the same function and the same set of parameters are used at every time step.



Recurrent Neural Network



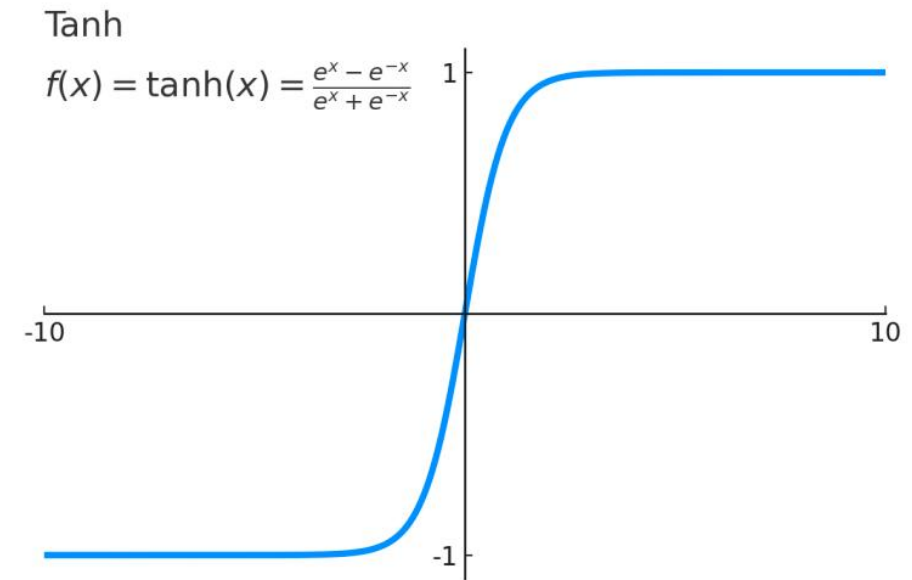
$$h_t = f_W(h_{t-1}, x_t)$$

$$y_t = g_{W_{hy}}(h_t)$$

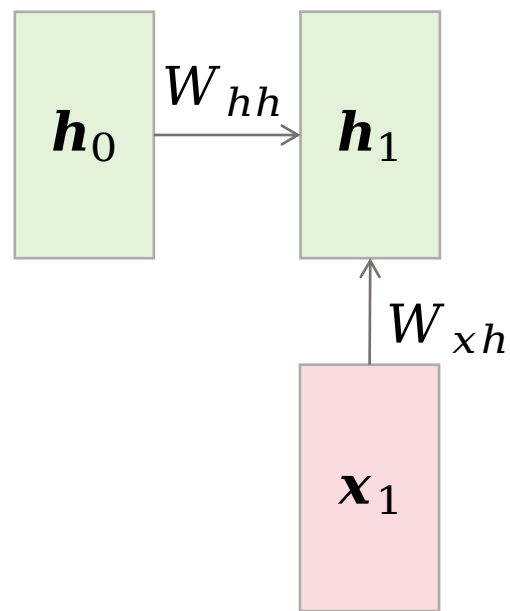


$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = g(W_{hy}h_t)$$

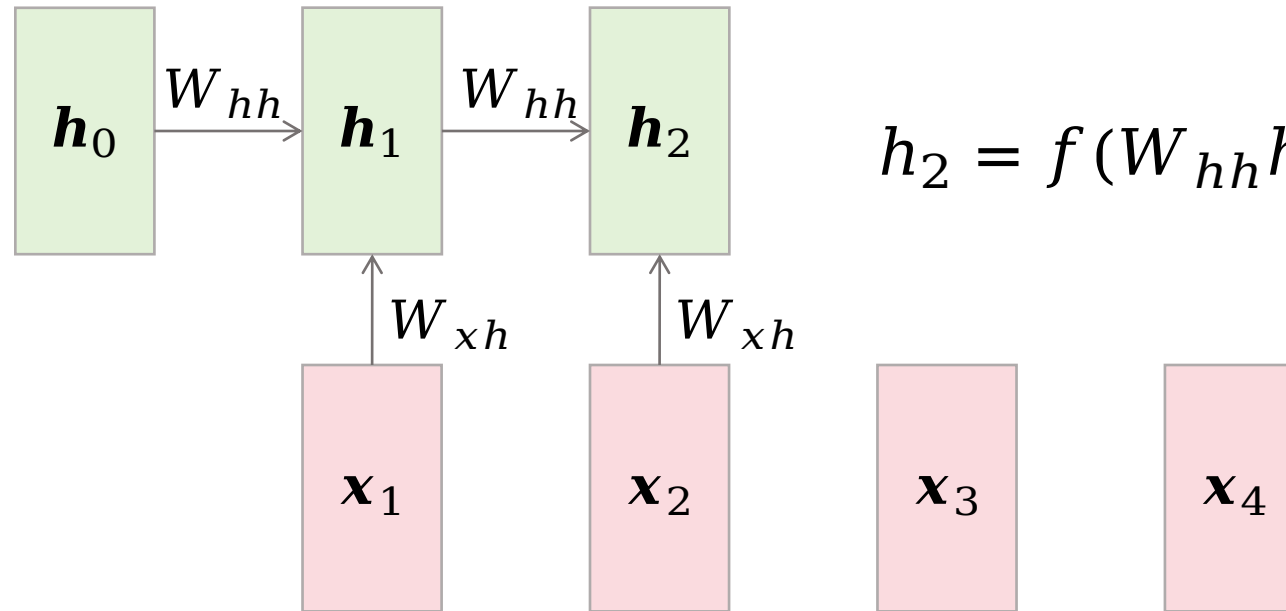


Recurrent Neural Network

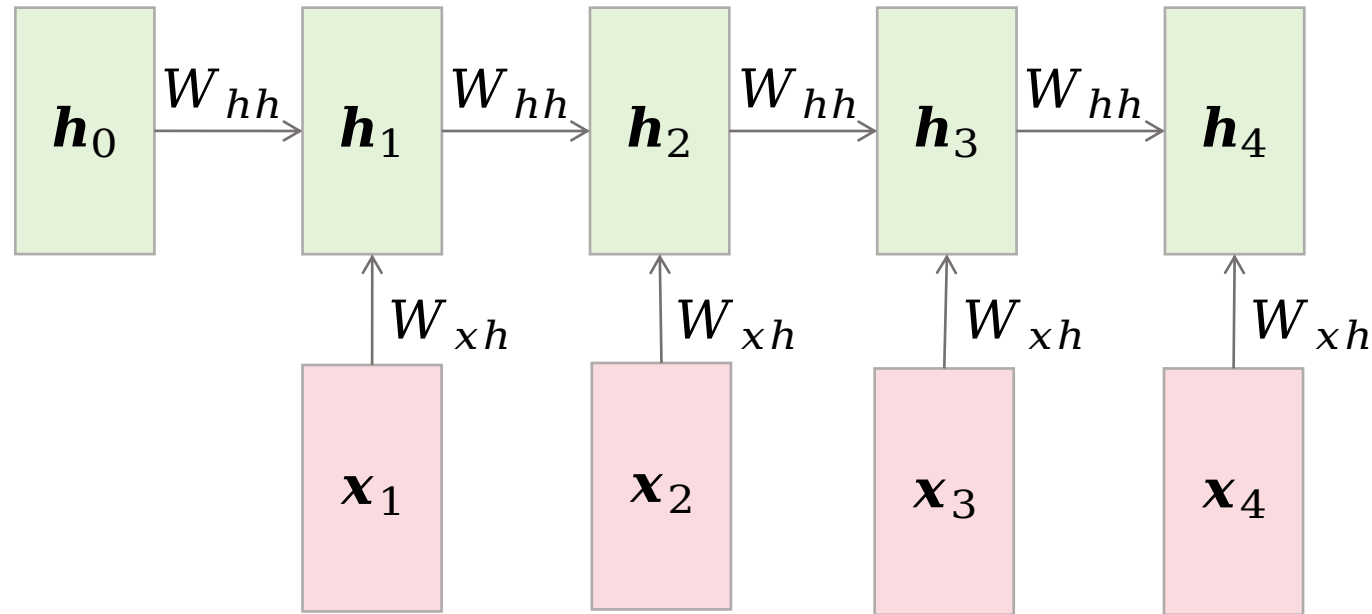


$$h_1 = f(W_{hh}h_0 + W_{xh}x_1)$$

Recurrent Neural Network



Recurrent Neural Network

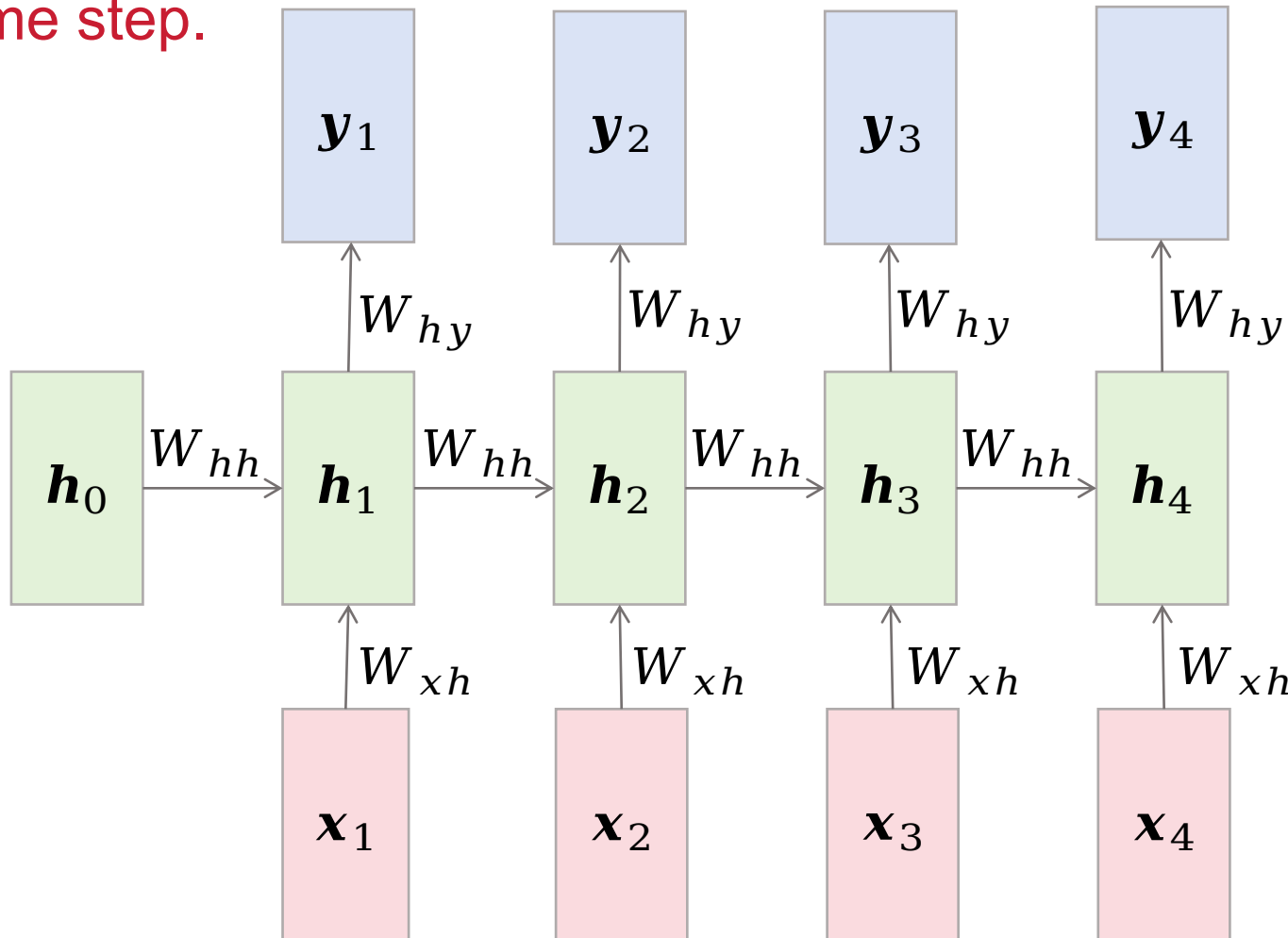


$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t)$$

Recurrent Neural Network



■ Notice: the same function and the same set of parameters are used at every time step.



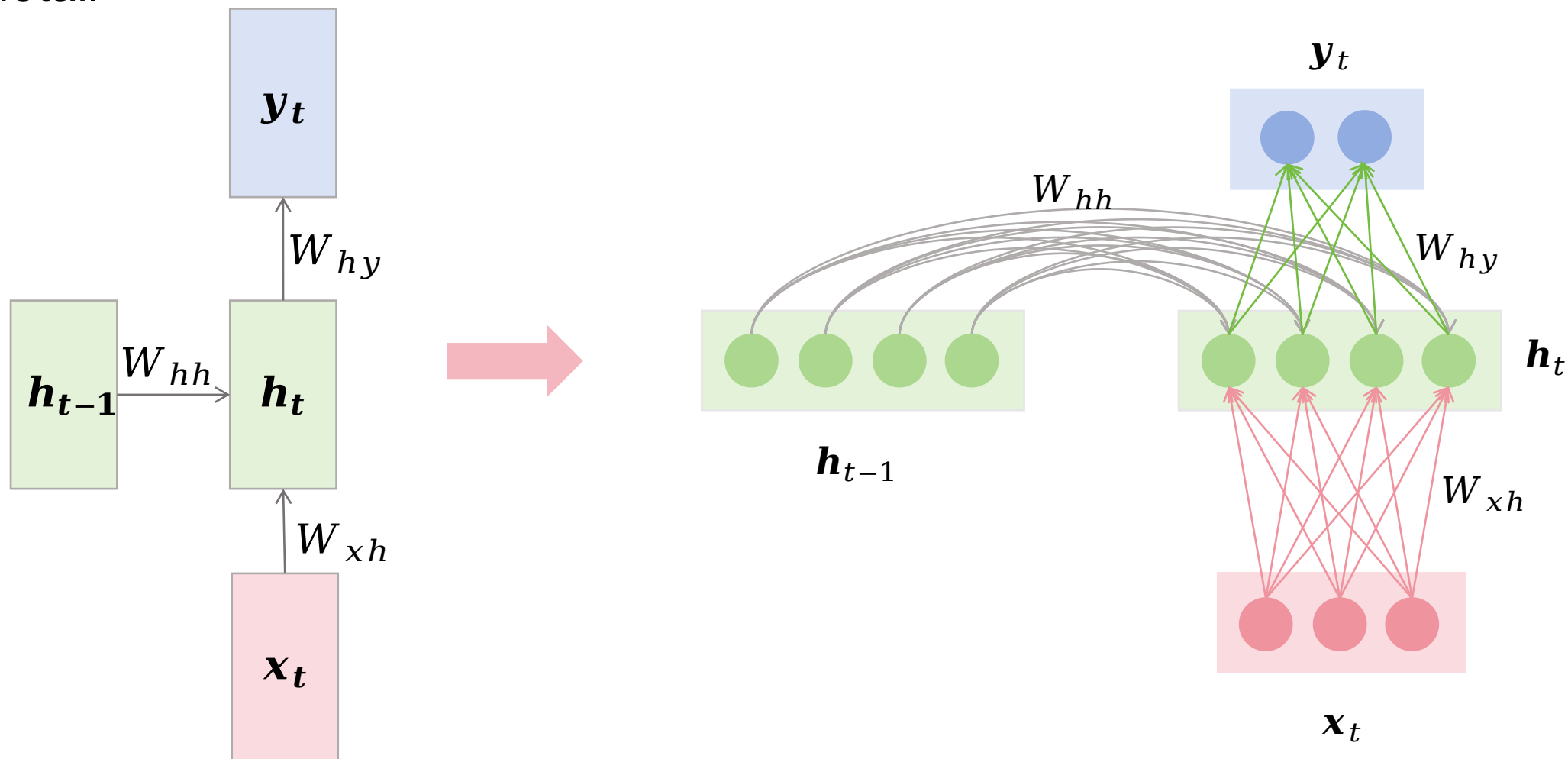
$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = g(W_{hy}h_t)$$

Recurrent Neural Network



■ detail





Pytorch Code

In PyTorch, nn.RNN returns two outputs:

- **output:** h_t from the last layer of the RNN for **each time step** t .
- **hidden:** the hidden state for the **last time step** ($t = \text{seq_len}$) for each layer.

$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = g(W_{hy}h_t)$$



depend on
your task



Pytorch
nn.RNN

```
import torch
import torch.nn as nn

torch.manual_seed(42)

input_size = 3
hidden_size = 6
num_layers = 1
seq_len = 5
batch_size = 2

rnn = nn.RNN(input_size=input_size,
              hidden_size=hidden_size,
              num_layers=num_layers,
              batch_first=True,
              bidirectional=False)

x = torch.randn(batch_size, seq_len, input_size) # [batch, seq_len, hidden_size]

output, hidden = rnn(x)

print("output shape:", output.shape) # [batch, seq_len, hidden_size]
print("hidden shape:", hidden.shape) # [num_layers, batch, hidden_size]

print(f"output tensor: \n", output)
print(f"hidden tensor: \n", hidden)
```


Pytorch Code



In PyTorch, nn.RNN returns two outputs:

- **output:** h_t from the last layer of the RNN for **each time step** t .
- **hidden:** the hidden state for the **last time step** ($t = \text{seq_len}$) for each layer.

```
input_size = 3
hidden_size = 6
num_layers = 1
seq_len = 5
batch_size = 2
```

```
output shape: torch.Size([2, 5, 6])
```

```
hidden shape: torch.Size([1, 2, 6])
```

```
output tensor:
```

```
tensor([[[ 0.7529,  0.6069,  0.0642, -0.5370,  0.8369, -0.3072],
         [-0.5958, -0.0602,  0.3485,  0.0806,  0.8381, -0.7257],
         [-0.0361,  0.3776, -0.0756,  0.0148,  0.5373, -0.7913],
         [ 0.2811,  0.2274, -0.3459,  0.1422,  0.5773, -0.6434],
         [-0.6648, -0.7274, -0.6063,  0.2437, -0.5671, -0.7225]]],
```

```
      t=1 [[ 0.5070,  0.0056,  0.3421, -0.0133,  0.5654, -0.1534],
      t=2 [ 0.1428,  0.0494,  0.6803,  0.0788,  0.8257, -0.4011],
      t=3 [-0.4864, -0.1580,  0.5481,  0.1581,  0.6646, -0.7719],
      t=4 [-0.0489,  0.4378,  0.0885, -0.1056,  0.5973, -0.8008],
      t=5 [ 0.2882,  0.5635, -0.4914, -0.2352,  0.7786, -0.7859]]],
```

```
      grad_fn=<TransposeBackward1>)
```

```
hidden tensor:
```

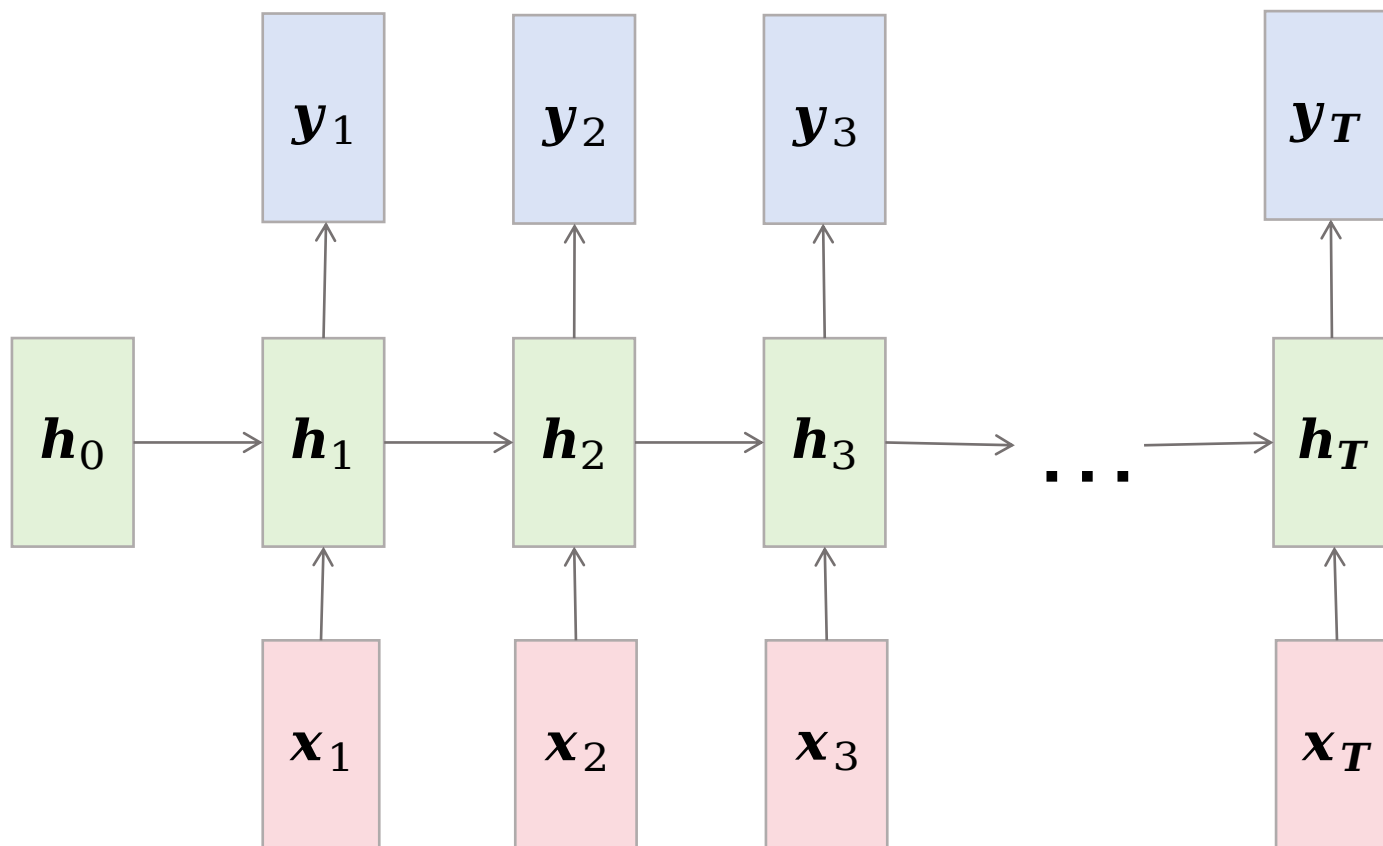
```
tensor([[[ -0.6648, -0.7274, -0.6063,  0.2437, -0.5671, -0.7225],
          [ 0.2882,  0.5635, -0.4914, -0.2352,  0.7786, -0.7859]]],
```

```
      grad_fn=<StackBackward0>)
```

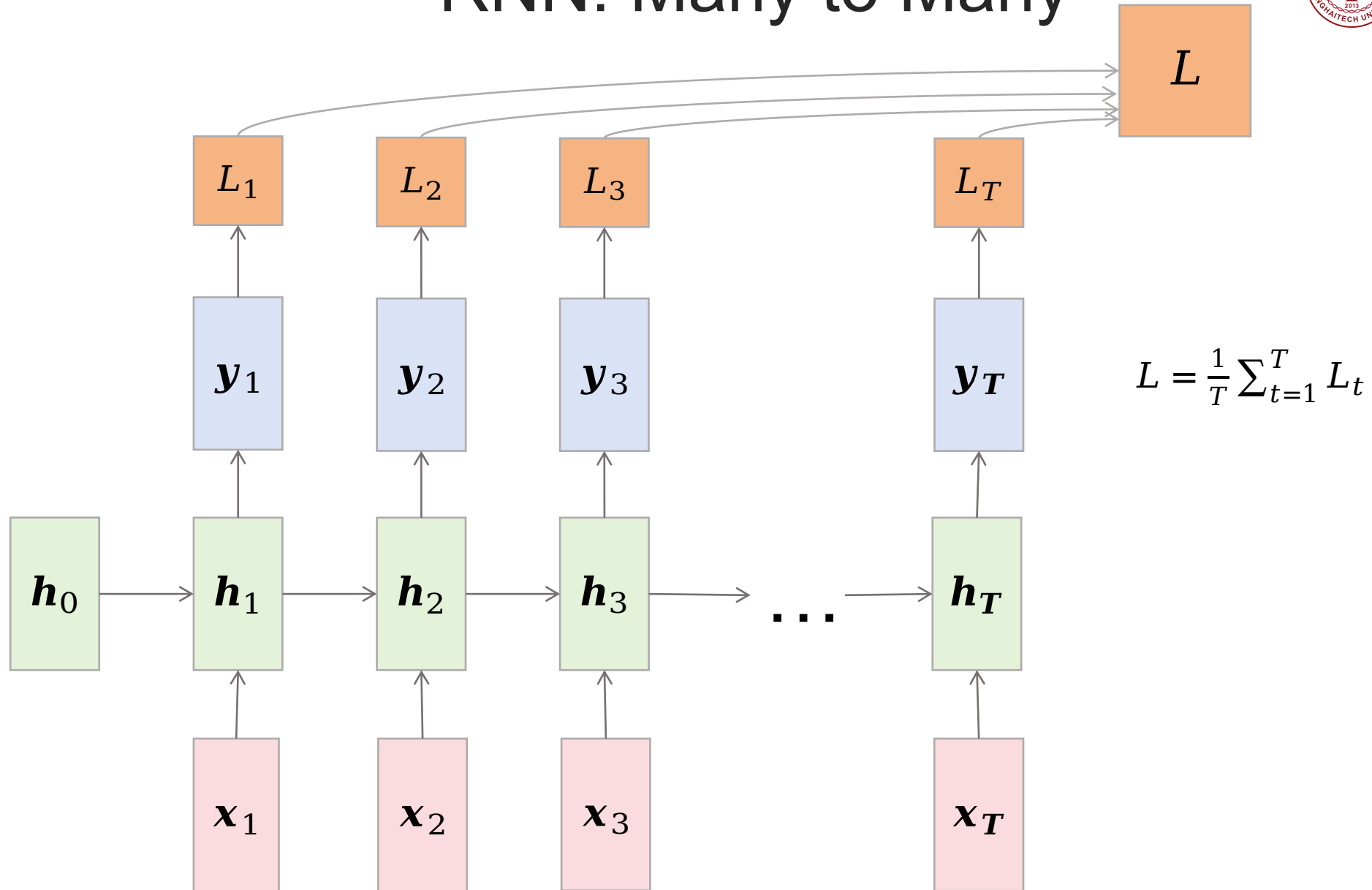
RNN: Many to Many



■ e.g. **sequence labeling**: sequence of words $\rightarrow$ sequence of labels



RNN: Many to Many

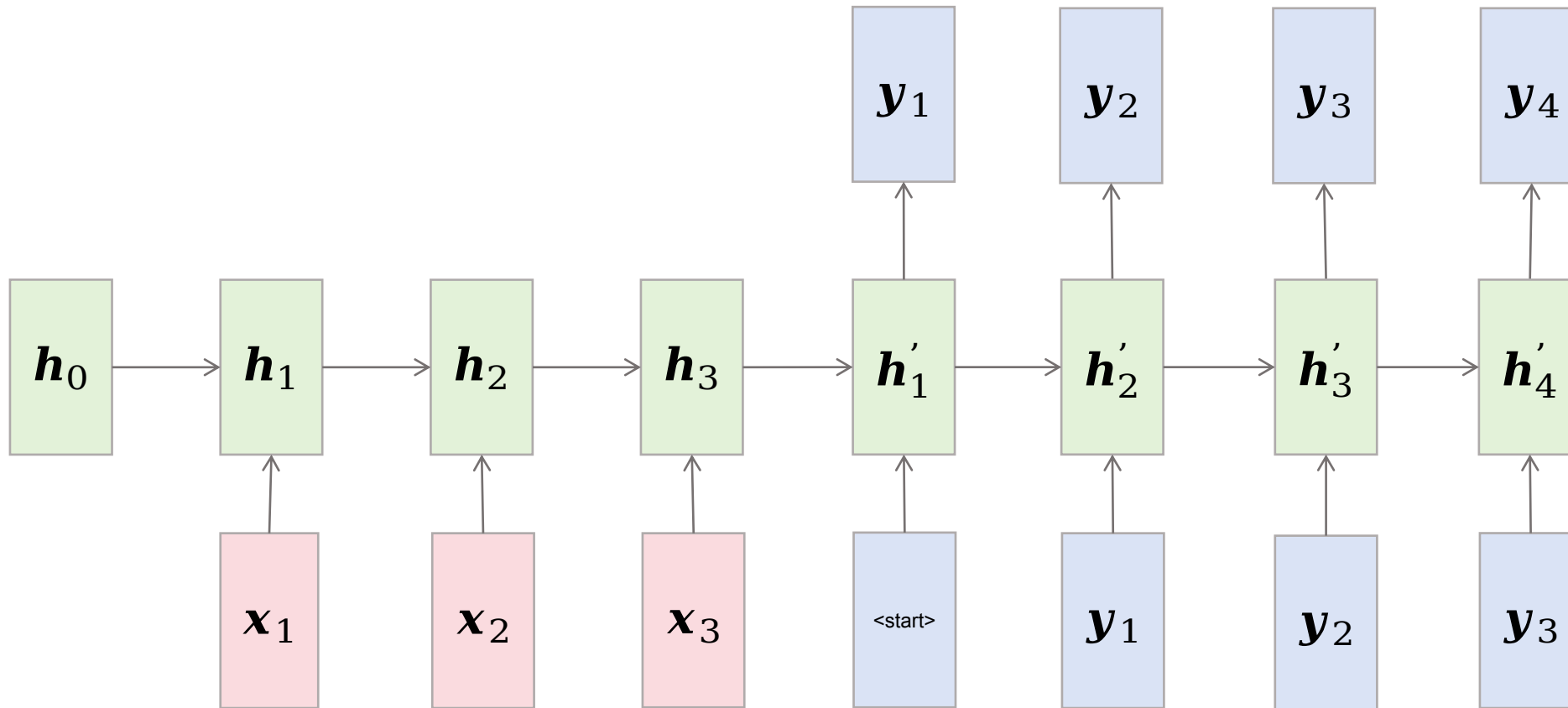


RNN: Many to Many



■ Seq2Seq or Encoder-Decoder

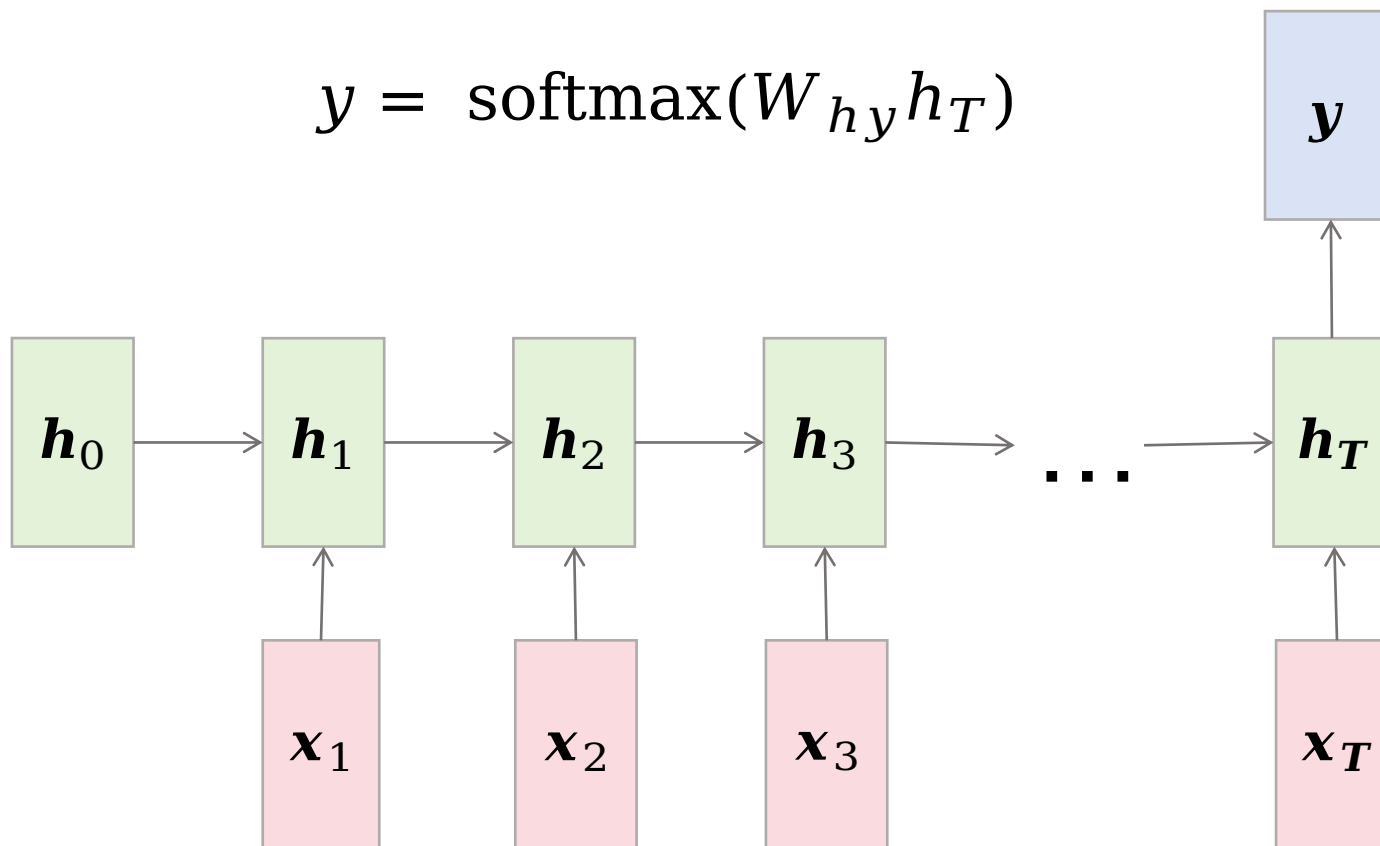
■ e.g. **machine translation**: sequence of words $\rightarrow$ sequence of words



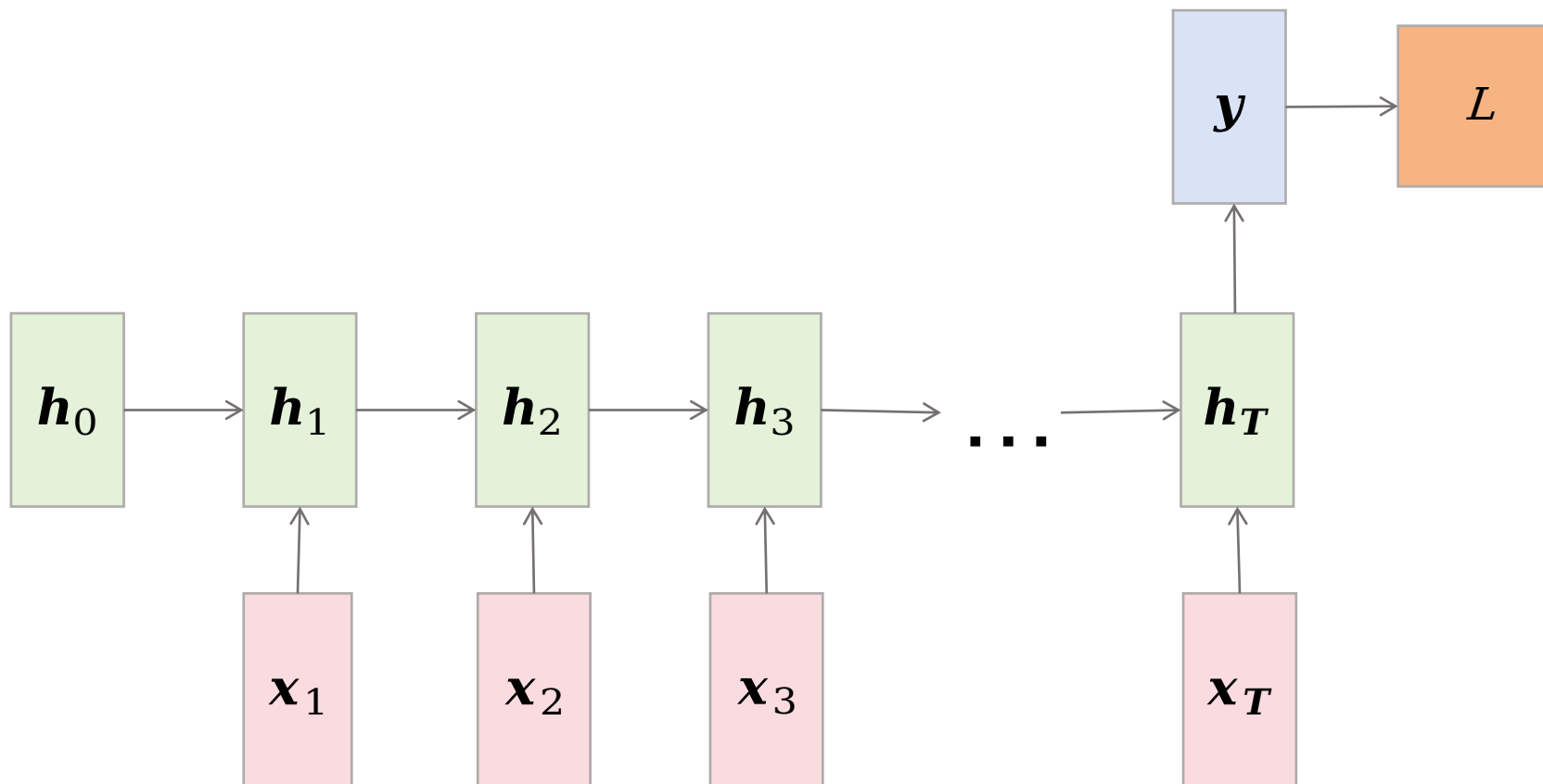
RNN: Many to One

■ e.g. **text classification**: sequence of words $\rightarrow$ label

$$y = \text{softmax}(W_{hy}h_T)$$



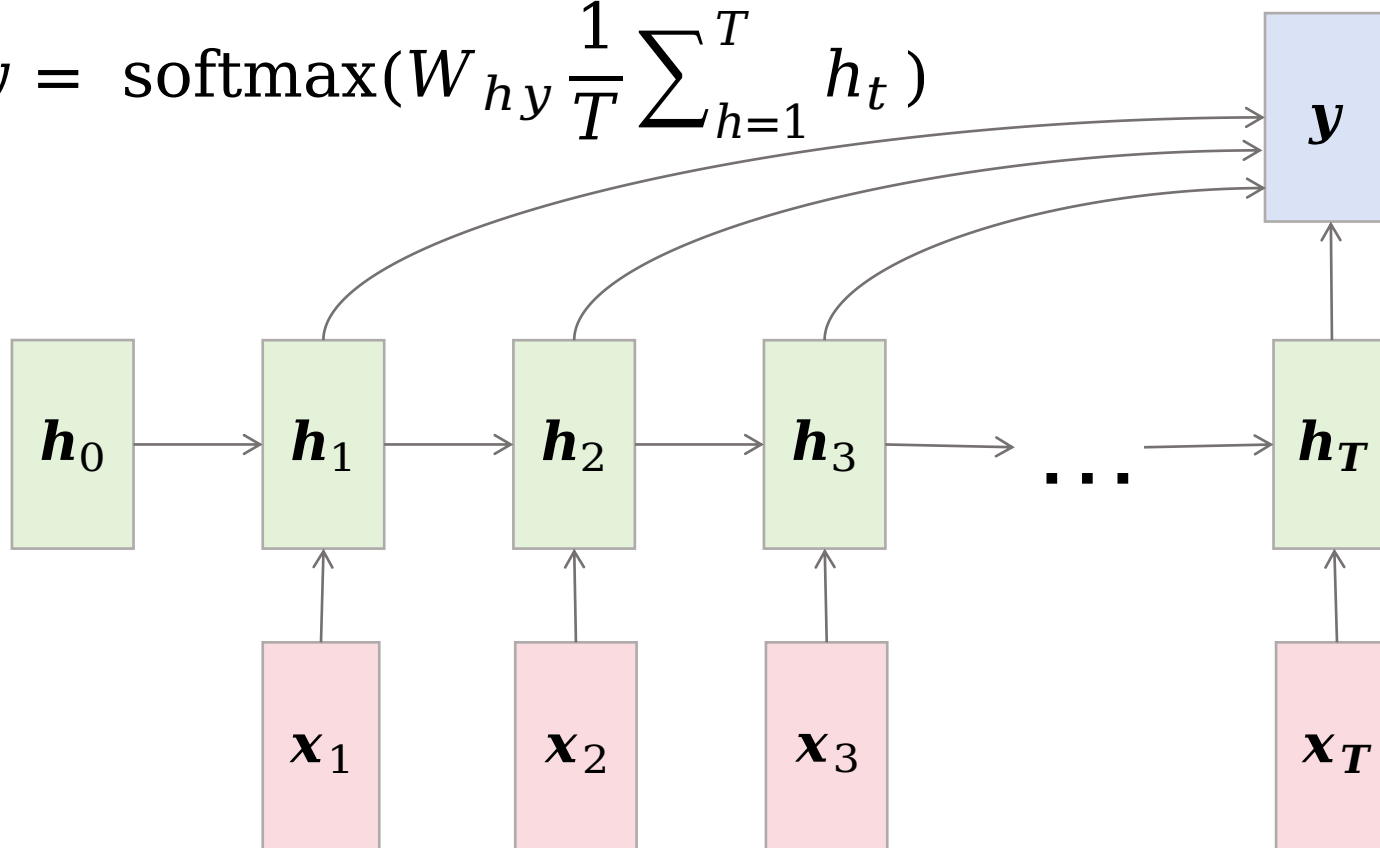
RNN: Many to One



RNN: Many to One

■ e.g. **text classification**: sequence of words $\rightarrow$ label

$$y = \text{softmax}(W_{hy} \frac{1}{T} \sum_{h=1}^T h_t)$$

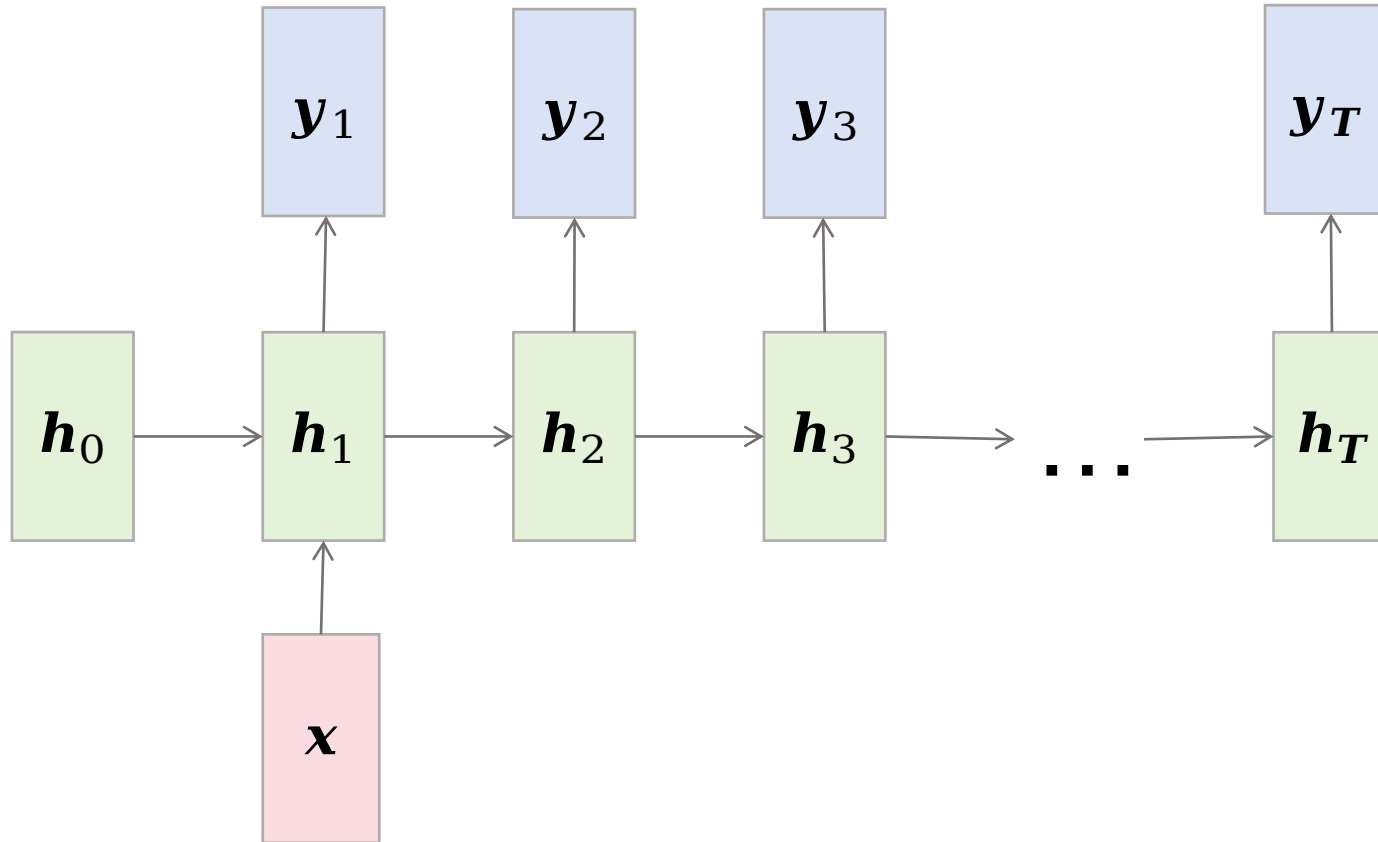


Take element-wise
max or mean of all
hidden states

RNN: One to Many



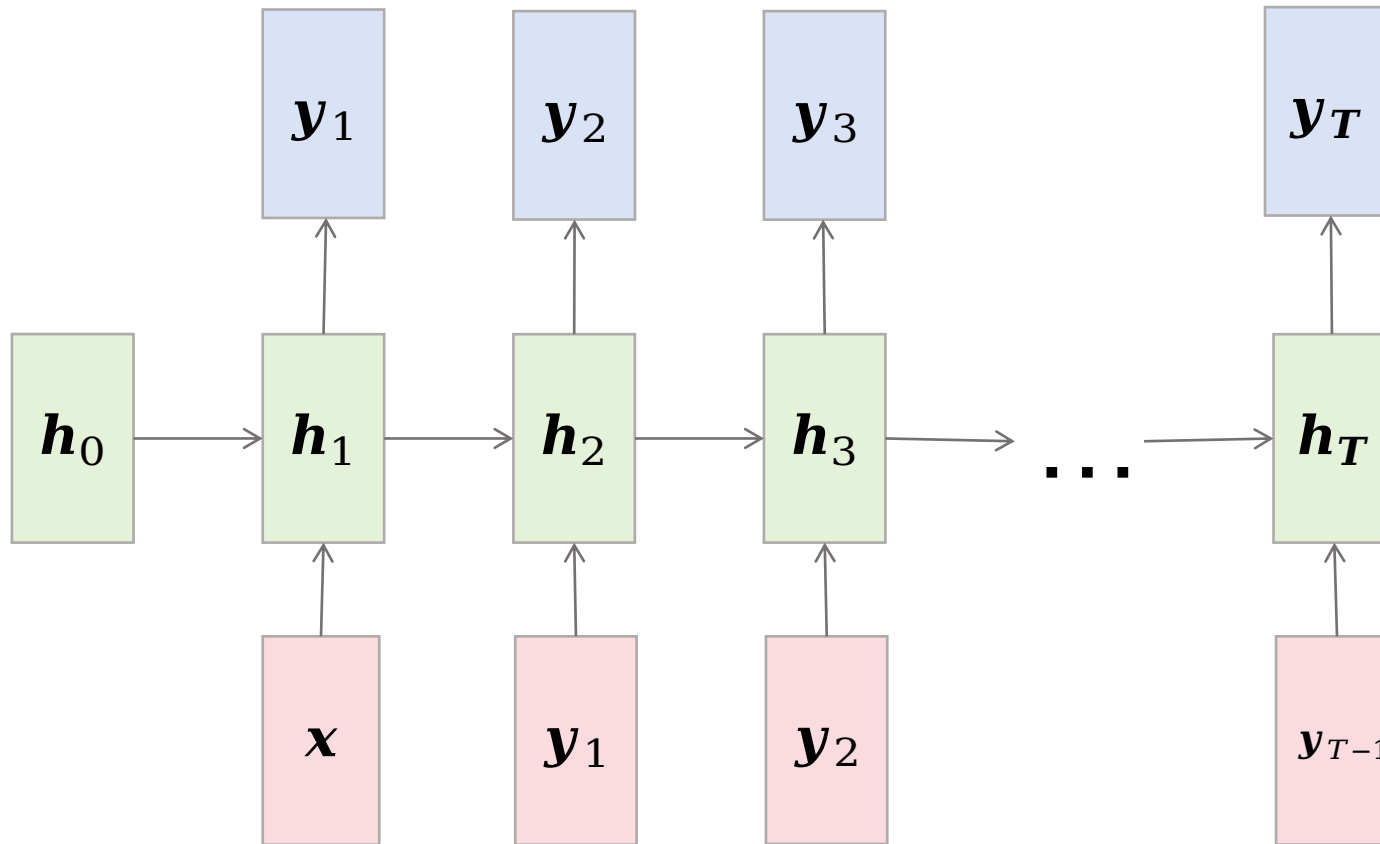
■ e.g. **image captioning**: image $\rightarrow$ sequence of words



RNN: One to Many



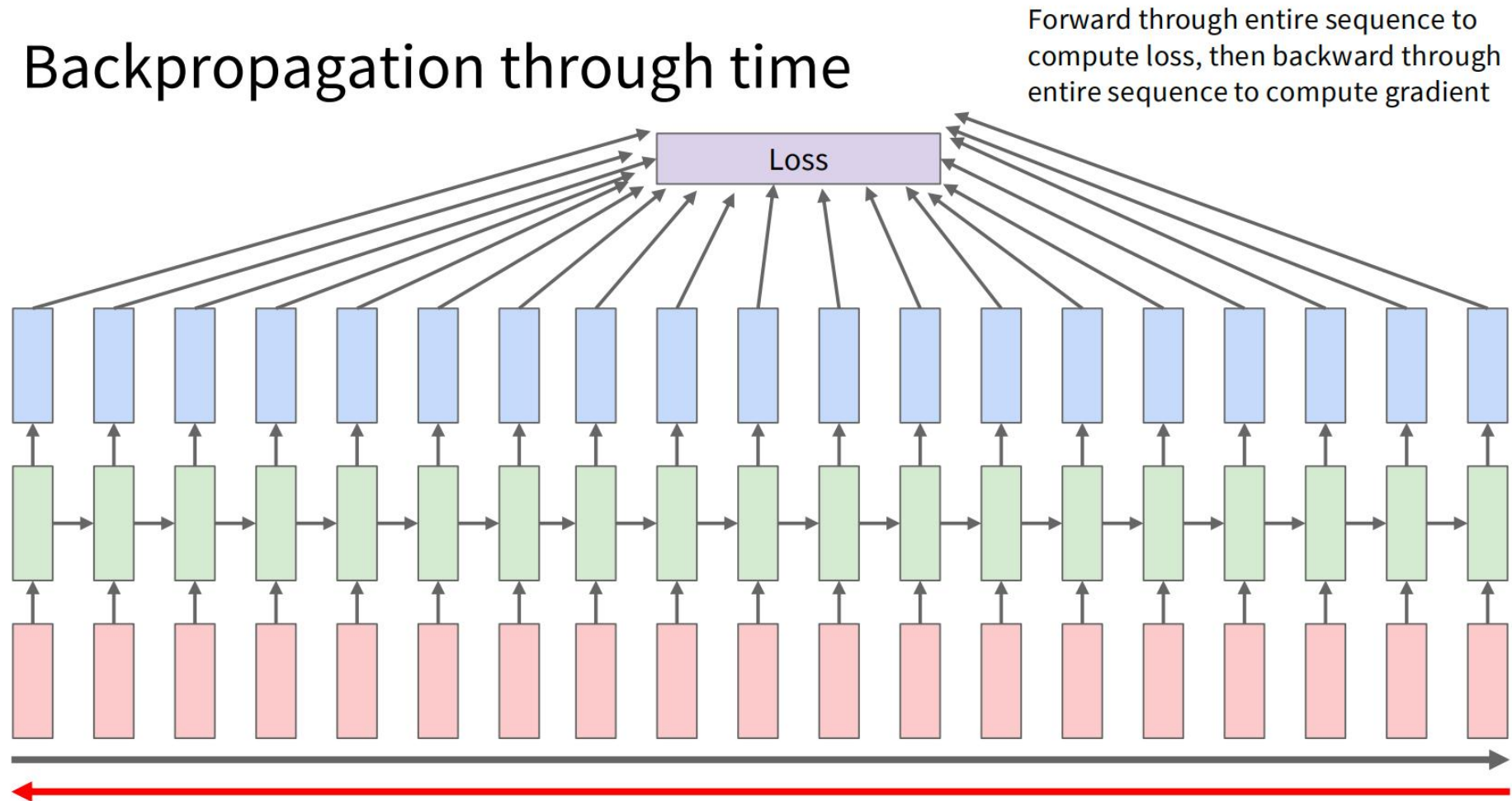
■ e.g. **image captioning**: image $\rightarrow$ sequence of words



Recurrent Neural Network



Backpropagation through time



source: https://cs231n.stanford.edu/slides/2025/lecture_7.pdf

Recurrent Neural Network



RNN: A family of neural networks that:

- Take sequential input of any length
- Apply the same weights on each step
- Can optionally produce output on each step

RNN tradeoffs



■ RNN Advantages:

- Can process any length of the input (no context length)
- Computation for step t can (in theory) use information from many steps back
- Model size does not increase for longer input
- The same weights are applied on every timestep, so there is symmetry in how inputs are processed.

■ RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

source: https://cs231n.stanford.edu/slides/2025/lecture_7.pdf

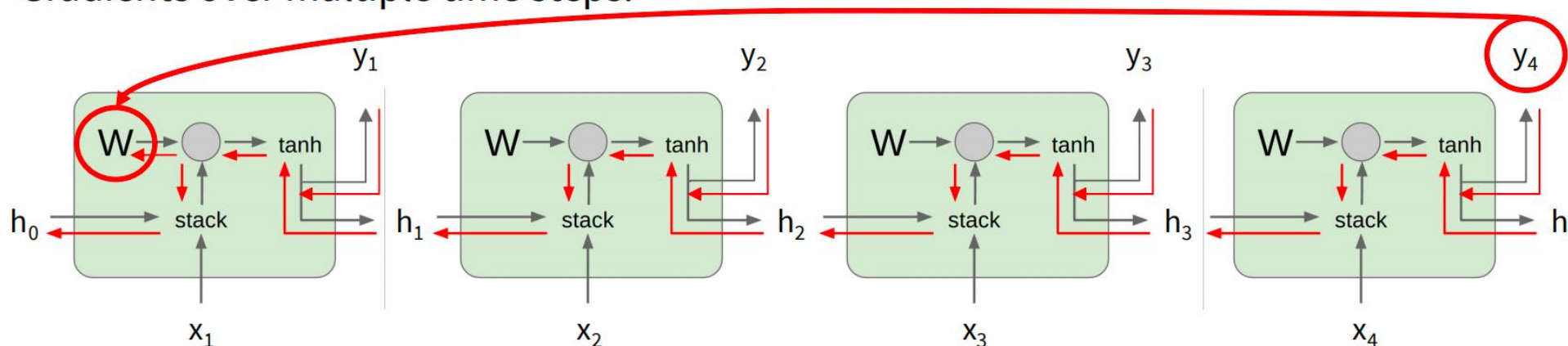
Problems with RNNs: Vanishing and Exploding Gradients



Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Largest singular value > 1:
Exploding gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value < 1:
Vanishing gradients

source: https://cs231n.stanford.edu/slides/2025/lecture_7.pdf

Exploding Gradient



■ Effect of exploding gradient

- cause bad updates: we take too large a step and reach a weird and bad parameter configuration (with large loss)

■ Solution for exploding gradient: **gradient clipping**

- if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

Algorithm 1 Pseudo-code for norm clipping

```
 $\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$   
if  $\|\hat{\mathbf{g}}\| \geq threshold$  then  
     $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$   
end if
```

“On the difficulty of training recurrent neural networks”, Pascanu et al, 2013. <http://proceedings.mlr.press/v28/pascanu13.pdf>
https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1254/slides_w25/cs224n-2025-lecture05-rnnlm.pdf

Vanishing Gradient



■ Effect of vanishing gradient

- If the gradient is small, the model can't learn long-distance dependencies. It's too difficult for the RNN to learn to preserve information over many timesteps.
- For example: 那只早上从院子里溜出去的黑猫，穿过小巷，跳过矮墙，还惊飞了几只麻雀，最后在邻居家的屋顶上晒太阳，看起来心情不错。它慵懒地伸了个懒腰，尾巴在瓦片上轻轻摆动，阳光洒在乌黑发亮的皮毛上，泛出一层暖融融的光晕。屋顶的风不大，刚好吹起它耳尖的一撮绒毛，它眯起眼睛，仿佛整个世界都安静了下来。

■ Solution for vanishing gradient

- Change RNN architecture -> LSTM

“On the difficulty of training recurrent neural networks”, Pascanu et al, 2013. <http://proceedings.mlr.press/v28/pascanu13.pdf>
https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1254/slides_w25/cs224n-2025-lecture05-rnnlm.pdf

Outline



■ Introduction to RNNs

- Formula and Architecture
- Sequential Tasks with RNNs
- Problems with RNNs

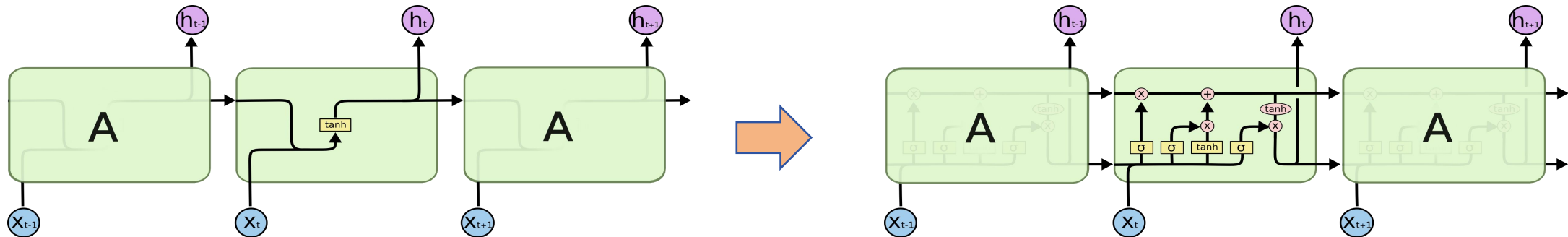
■ Long Short-Term Memory (LSTM)

■ Seq2Seq Model and Attention

Long Short Term Memory (LSTM)



■ LSTM is one of RNN variants



- Hochreiter and Schmidhuber, 1997. Long short-term memory. <https://www.bioinf.jku.at/publications/older/2604.pdf>
- Gers, Schmidhuber, and Cummins, 2000. Learning to Forget: Continual Prediction with LSTM. <https://dl.acm.org/doi/10.1162/089976600300015015>
- Graves, Fernandez, Gomez, and Schmidhuber, 2006. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural nets. https://www.cs.toronto.edu/~graves/icml_2006.pdf
- <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long Short Term Memory (LSTM)

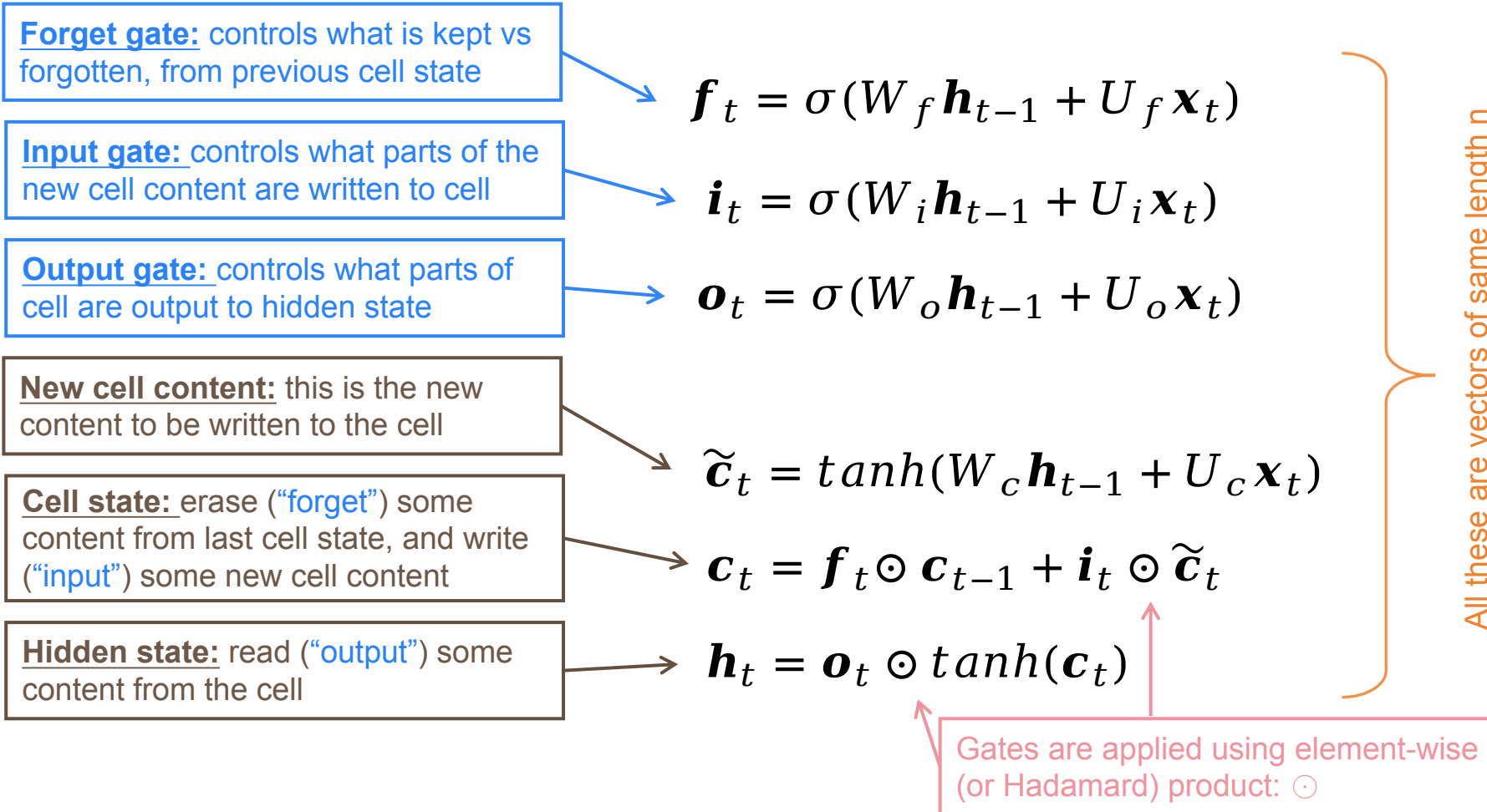


- On step t , there is a **hidden state** $h(t)$ and a **cell state** $c(t)$
 - Both are vectors length n
 - The cell stores **long-term information**
 - The LSTM can read, erase, and write information from the cell
- The selection of which information is erased/written/read is controlled by three corresponding **gates**
 - The gates are also vectors of length n
 - On each timestep, each element of the gates can be **open** (1), **closed** (0), or somewhere in-between
 - The gates are **dynamic**: their value is computed based on the current context

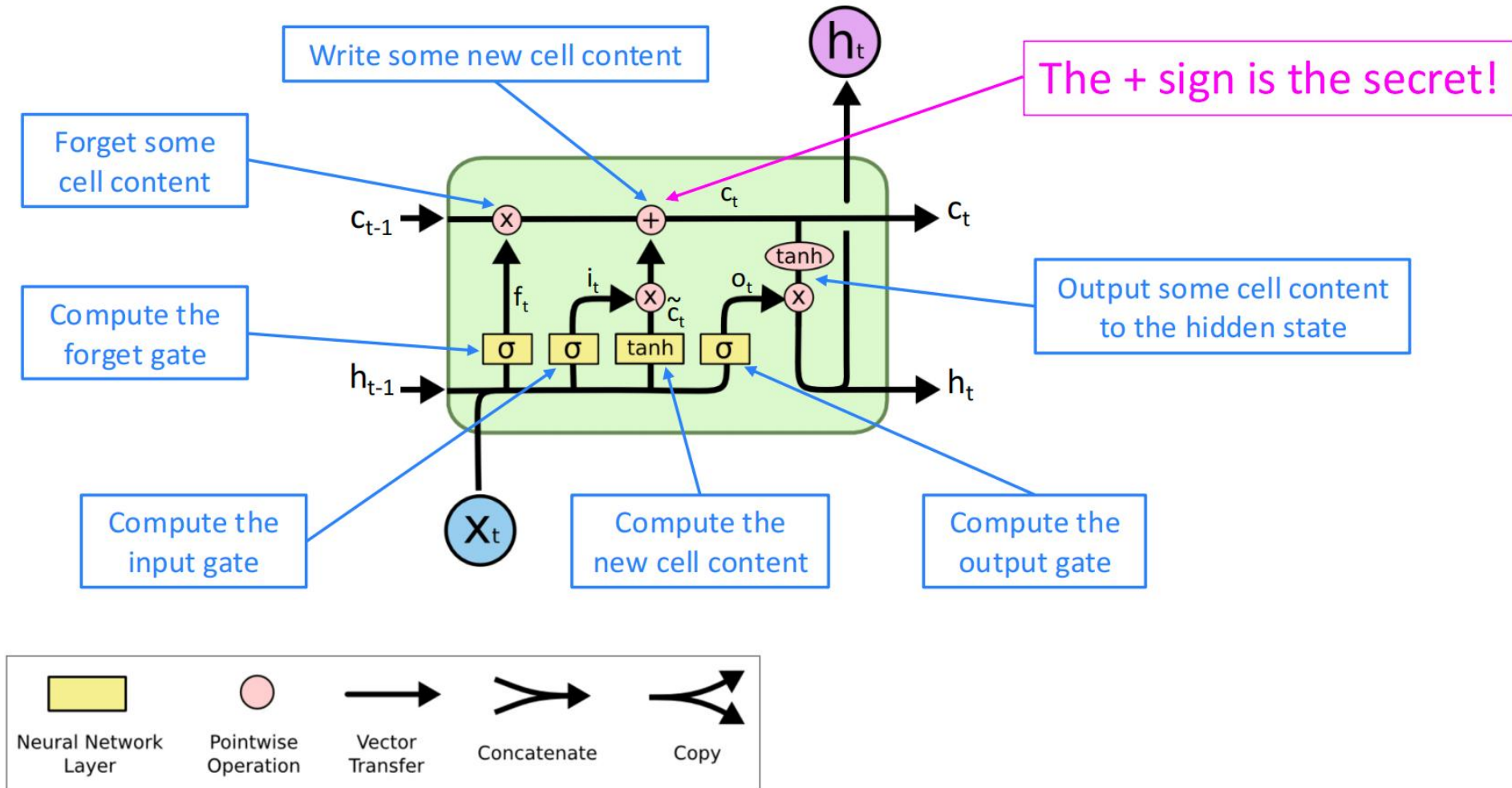
Long Short Term Memory (LSTM)



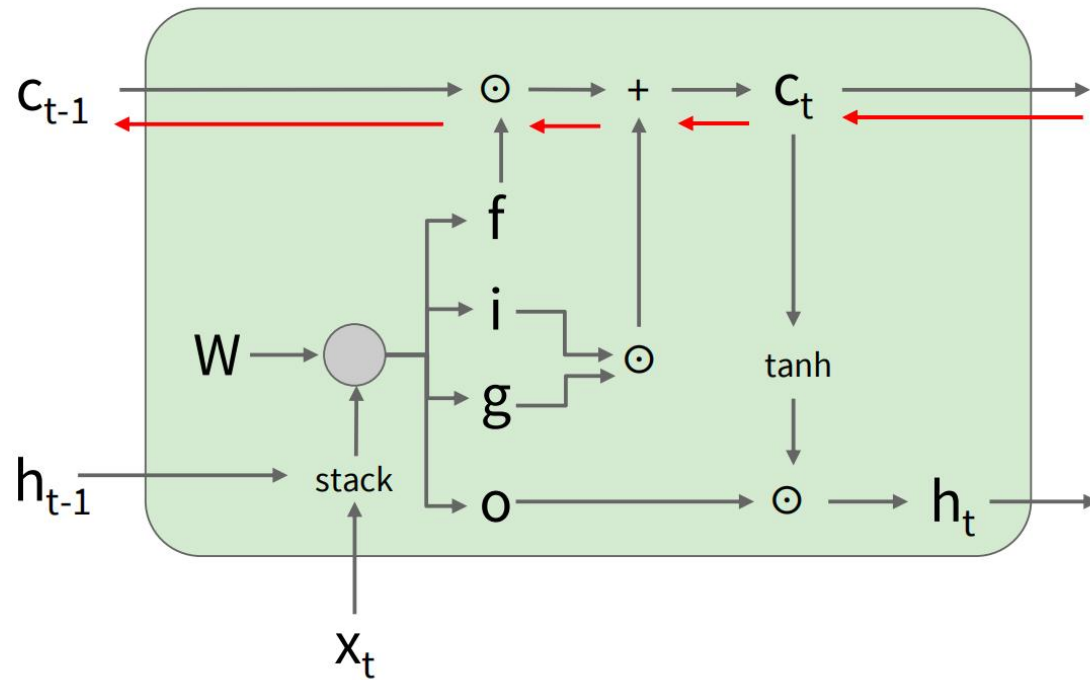
We have a sequence of inputs $\mathbf{x}_t$, and we will compute a sequence of hidden states $\mathbf{h}_t$ and cell states $\mathbf{c}_t$. On timestep t :



Long Short Term Memory (LSTM)



LSTM Gradient Flow



$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

Backpropagation from c_t to c_{t-1} only
elementwise multiplication by f , no matrix
multiply by W .

Do LSTMs solve the vanishing gradient problem?



- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps.
 - e.g. if the $f = 1$ and the $i = 0$, then the information of that cell is preserved indefinitely.
 - By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix W_h that preserves info in hidden state.
- LSTM doesn't guarantee that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies.

LSTMs: real-world success

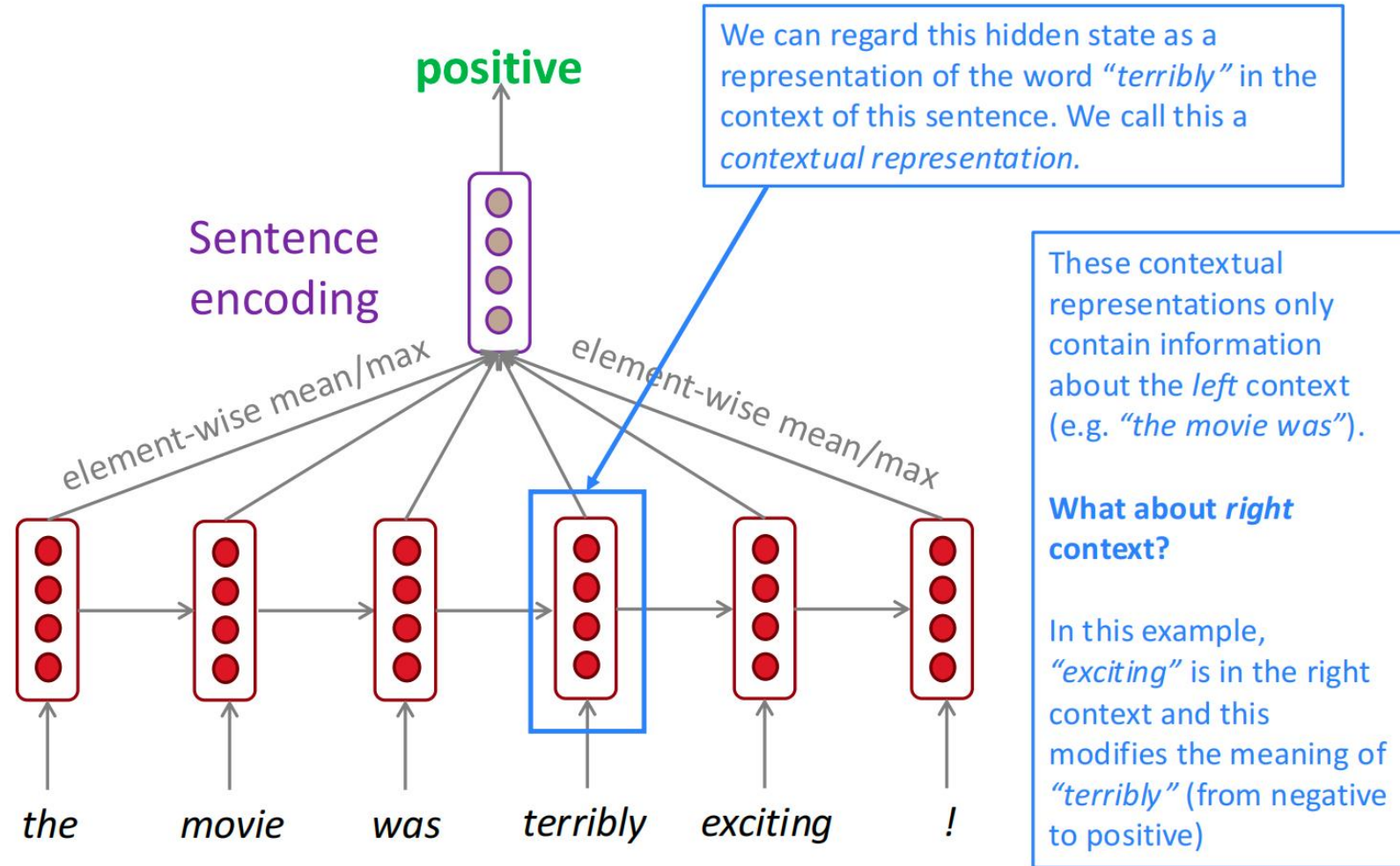


- In 2013–2015, LSTMs started achieving state-of-the-art results
 - Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
 - LSTMs became the dominant approach for most NLP tasks
- Recently, Transformers have become dominant for all tasks

Bidirectional RNNs: Motivation



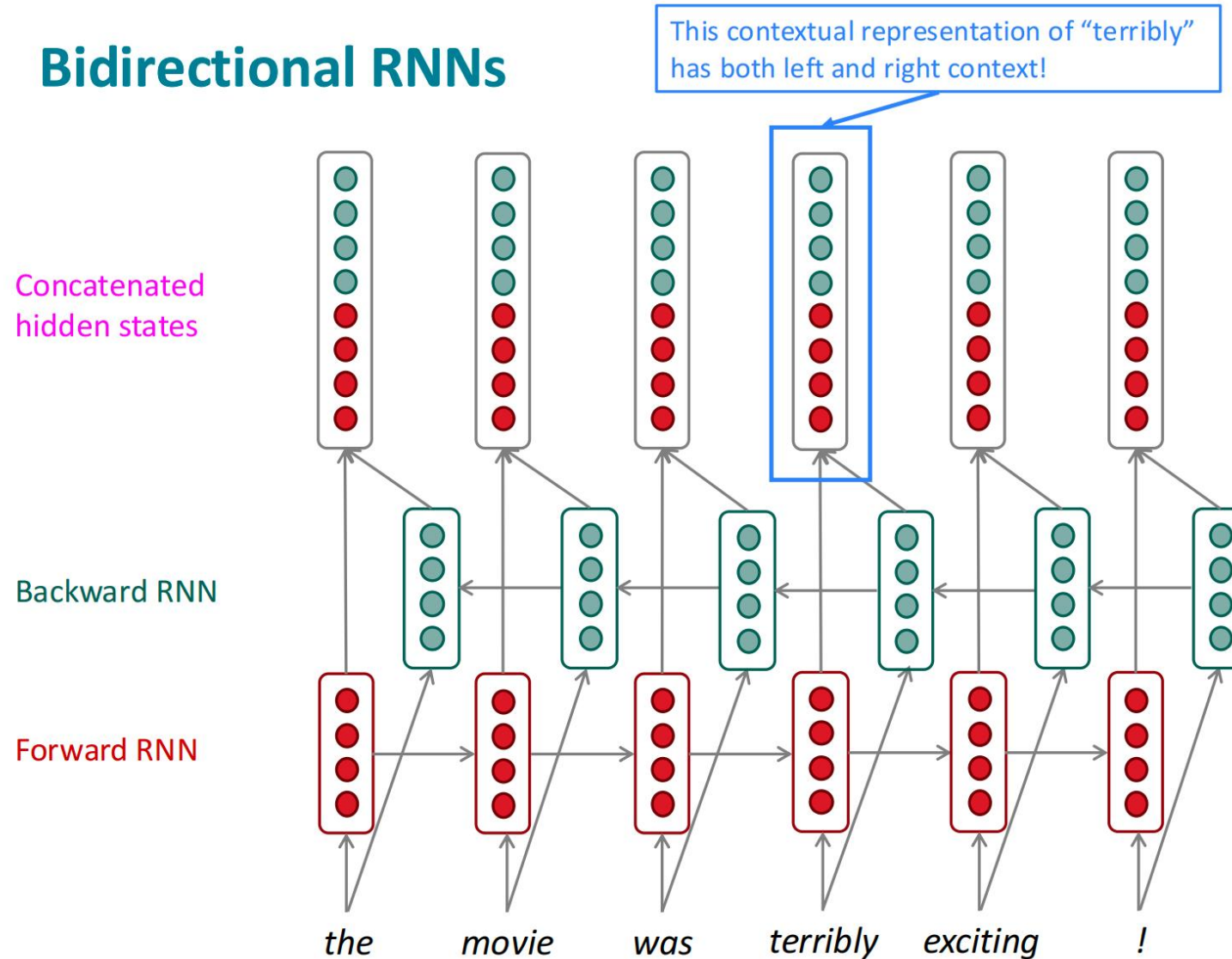
Task: Sentiment Classification



Bidirectional RNNs



Bidirectional RNNs



When to use bidirectional RNNs



Note: bidirectional RNNs are only applicable if you have access to the **entire input sequence**.

■ When to use:

- ☐ Sequence Labeling: e.g. Named Entity Recognition (NER), Part-of-Speech (POS) Tagging
- ☐ Text Classification: e.g. Sentiment Analysis
- ☐ Machine Translation: When encoding source sentences
- ☐ Handwriting Recognition

■ When not to use:

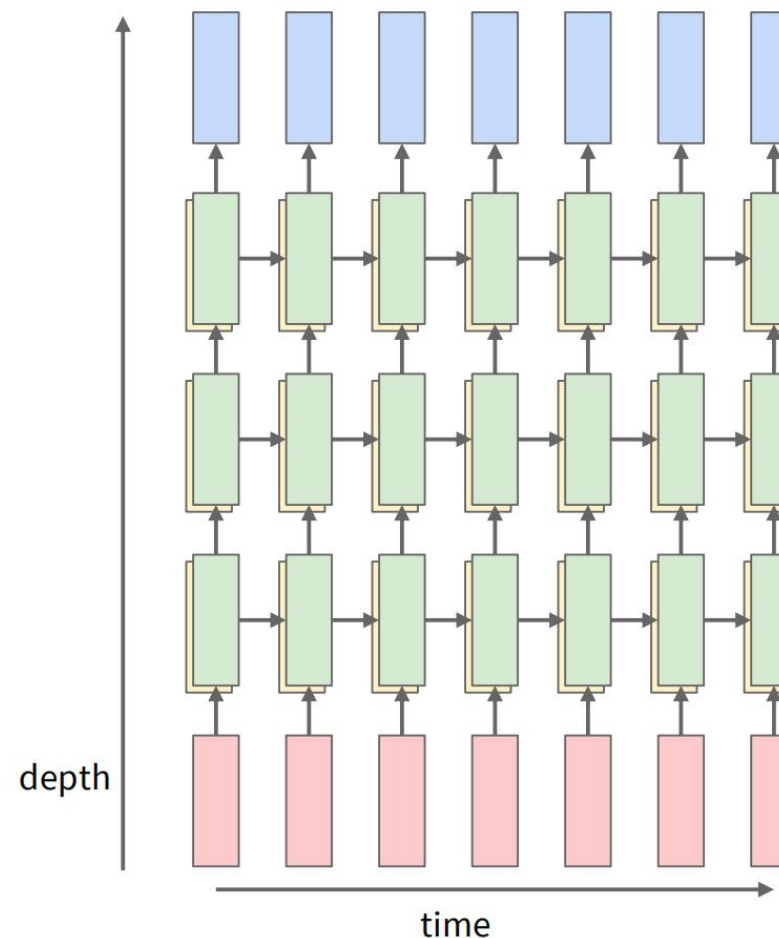
- ☐ Language Modeling: In LM you only have left context available
- ☐ Real-time prediction: Cannot be used when future data isn't available yet

BERT (Bidirectional Encoder Representations from Transformers) is a powerful pretrained contextual representation system **built on bidirectionality**.

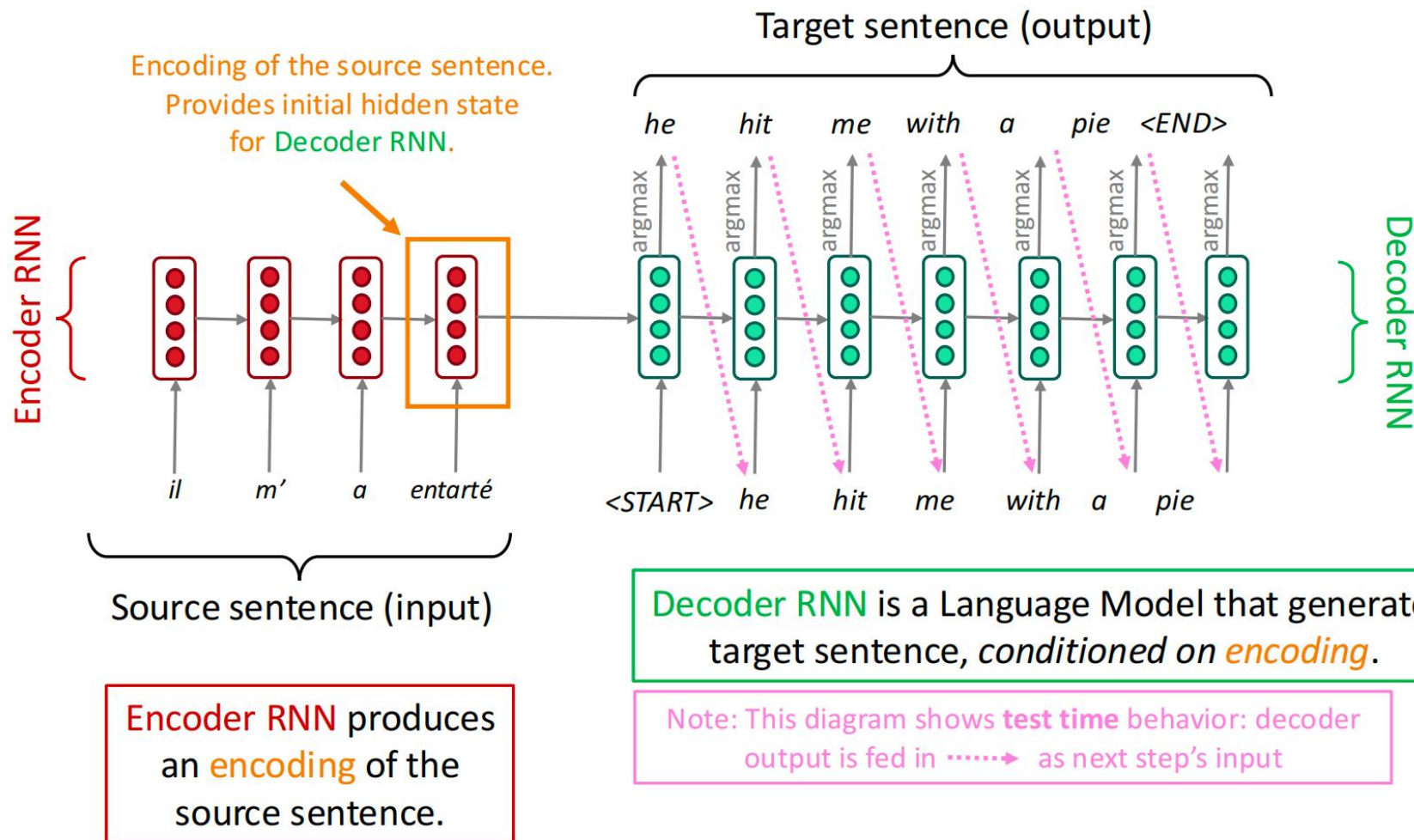
Multi-layer RNNs



- Multi-layer RNNs allow a network to compute more complex representations. They work better than just have one layer of high-dimensional encodings!
- High-performing RNNs are usually multi-layer (but aren't as deep as convolutional or feed-forward networks)
- For many tasks, 2-3 layers of RNNs are sufficient. Often 2 layers is a lot better than 1, and 3 might be a little better than 2
- Transformer-based networks (e.g., BERT) are usually deeper, like 12 or 24 layers.



sequence-to-sequence model



https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1254/slides_w25/cs224n-2025-lecture06-fancy-rnn.pdf
Sutskever et al, 2014. Sequence to sequence learning with neural networks. <https://arxiv.org/pdf/1409.3215>

sequence-to-sequence model



■ The general notion here is an **encoder-decoder** model

- One neural network takes input and produces a neural representation
- Another network produces output based on that neural representation
- If the input and output are sequences, we call it a **seq2seq** model

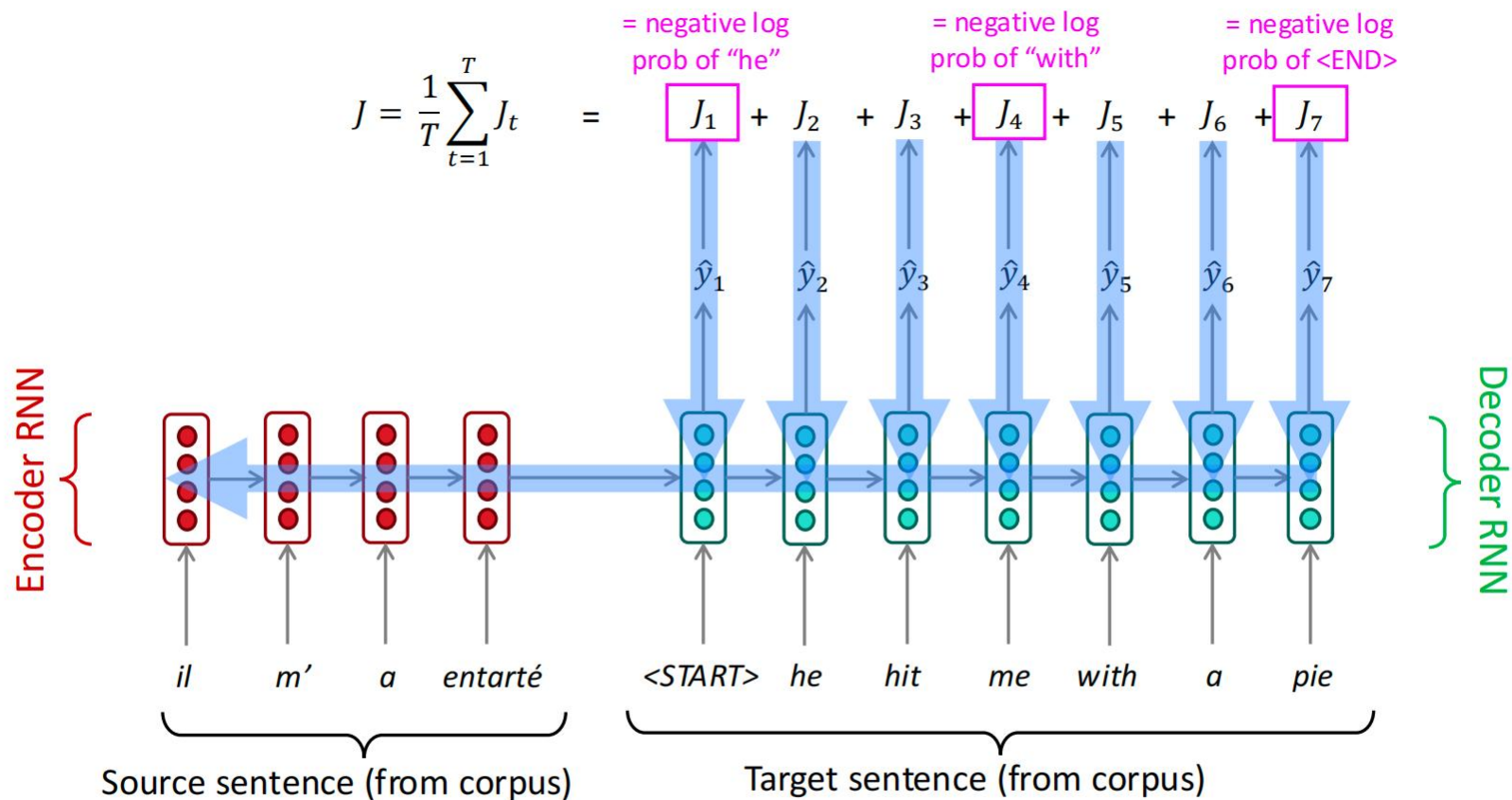
■ Many NLP tasks can be phrased as sequence-to-sequence:

- **Machine Translation** (source sentence $\rightarrow$ target sentence)
- **Summarization** (long text $\rightarrow$ short text)
- **Dialogue** (previous utterances $\rightarrow$ next utterance)
- **Parsing** (input text $\rightarrow$ output parse as sequence)
- **Code Generation** (natural language $\rightarrow$ code)

sequence-to-sequence model

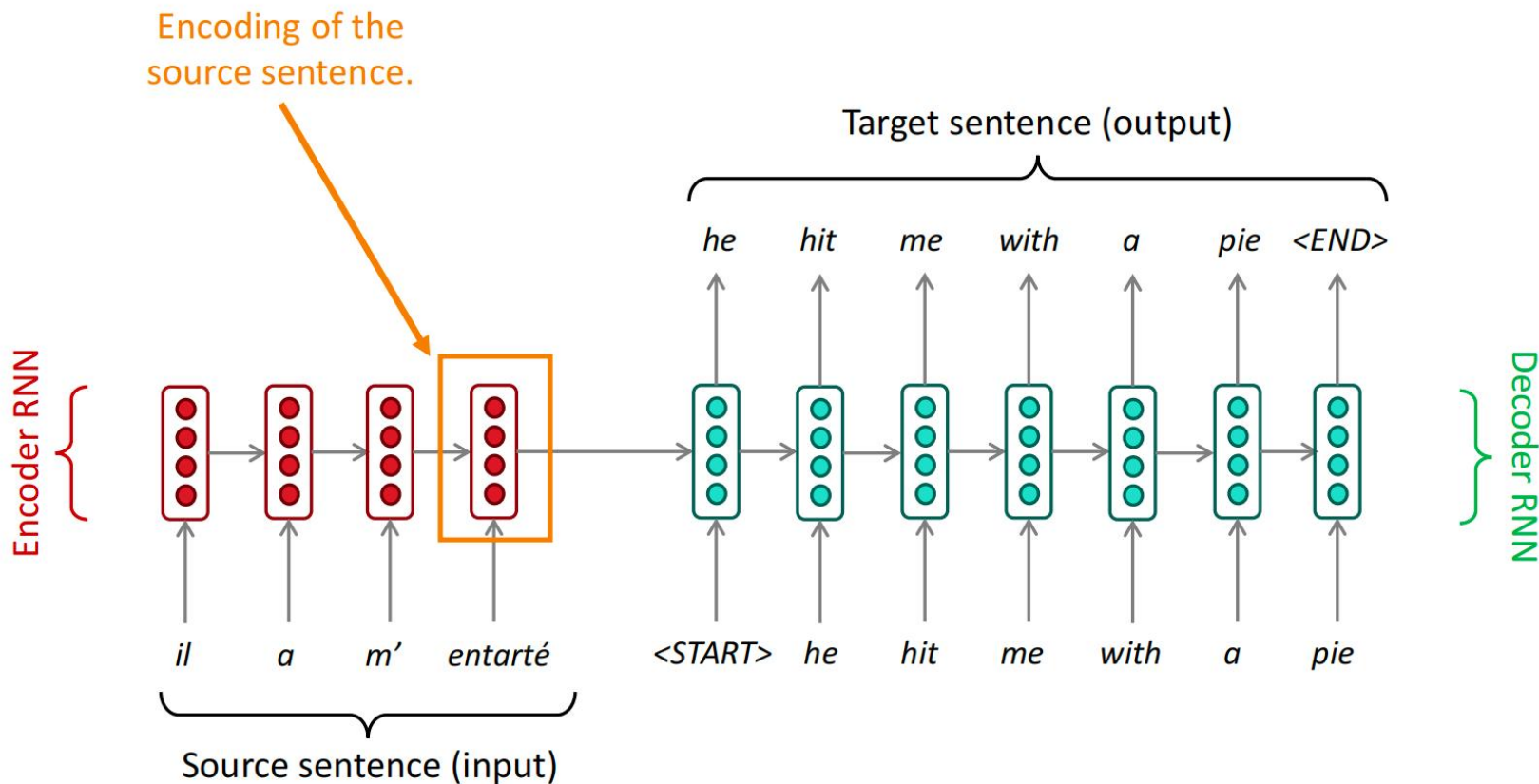


Training process



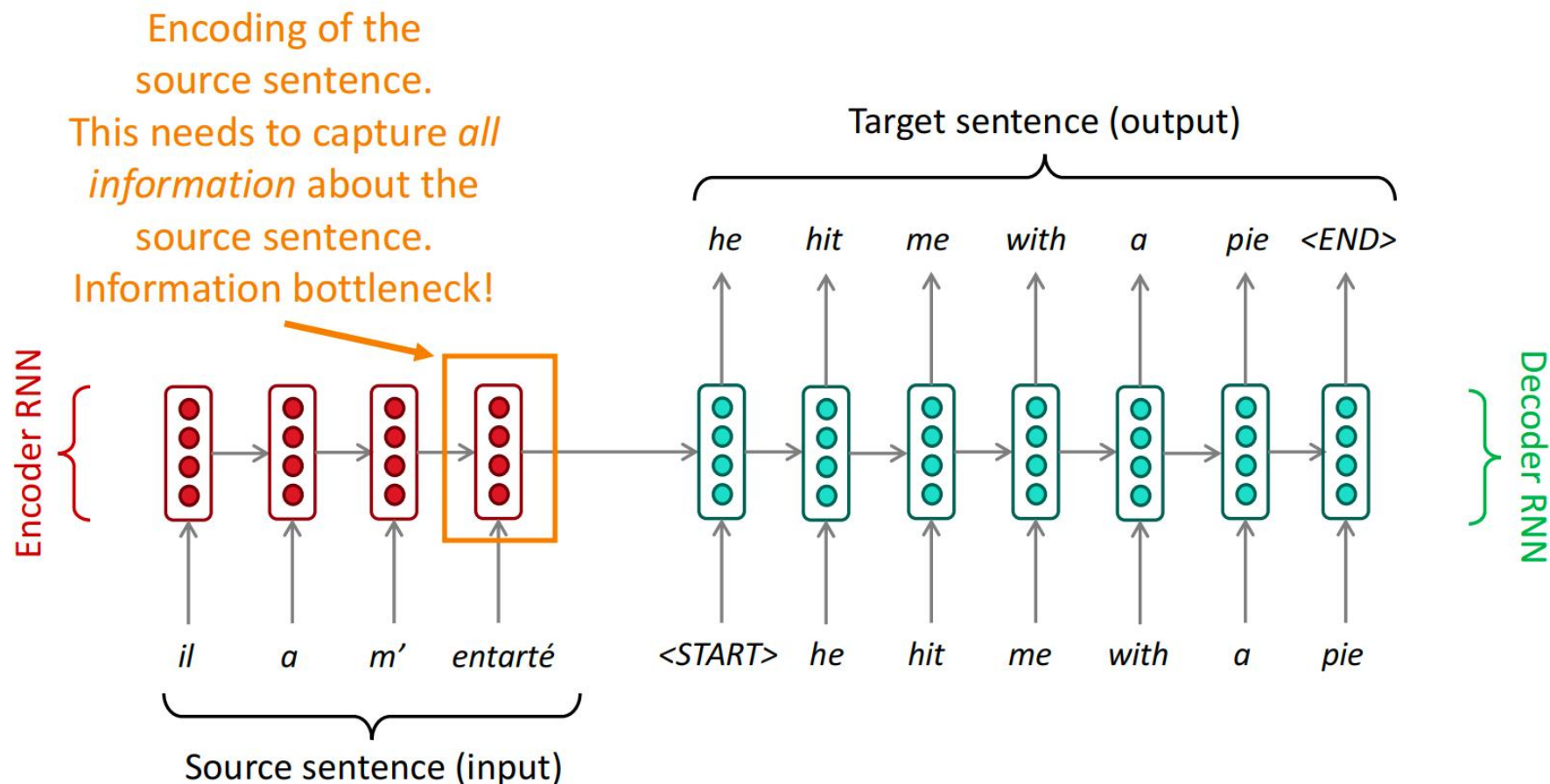
Seq2seq is optimized as a **single system**. Backpropagation operates "end-to-end".

Sequence-to-sequence: the bottleneck problem



Problems with this architecture?

Sequence-to-sequence: the bottleneck problem



Attention



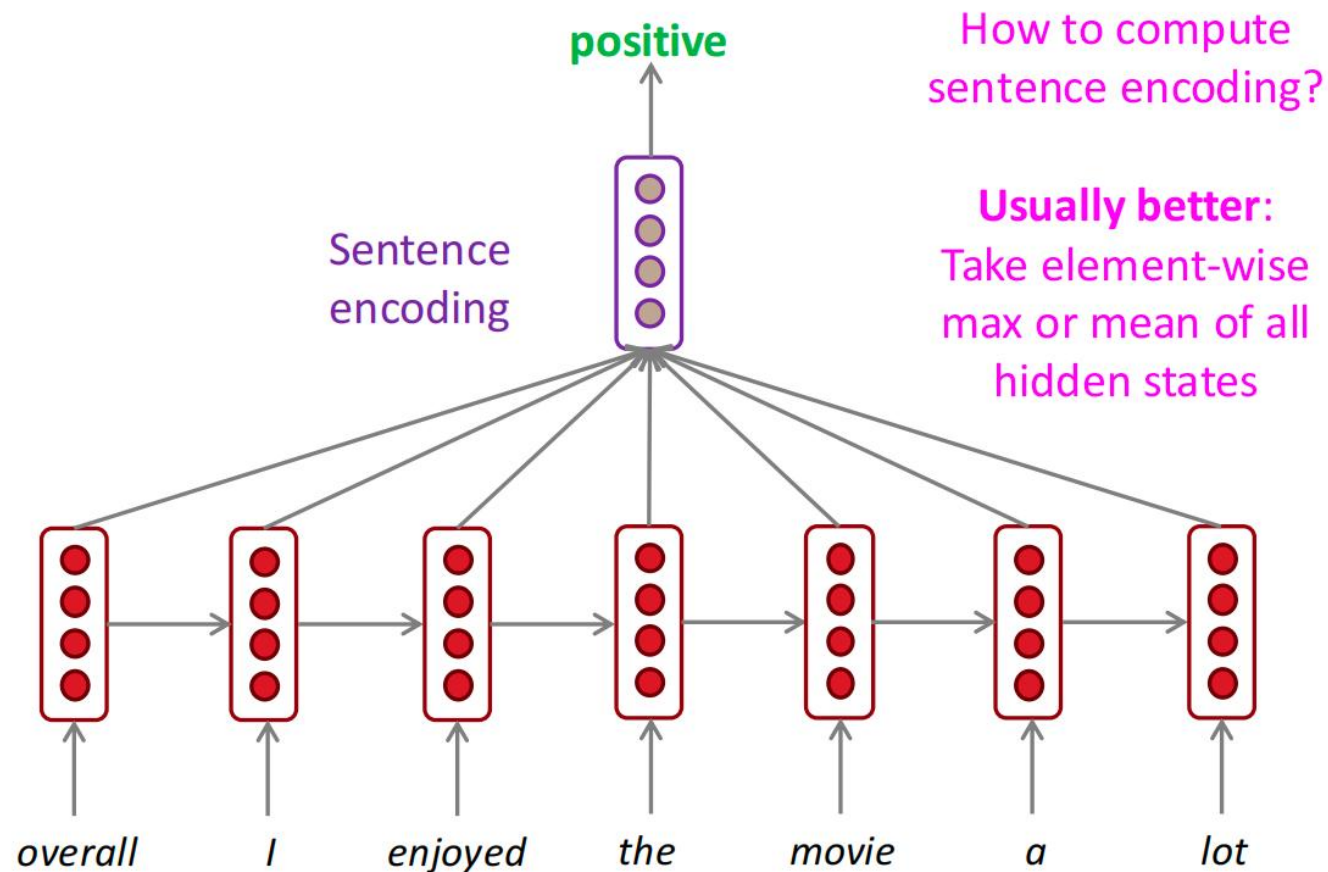
- **Attention** provides a solution to the bottleneck problem.
- Core idea: on each step of the decoder, use direct connection to the encoder to focus on a particular part of the source sequence



The starting point: mean-pooling for RNNs



Starting point: a very basic way of ‘passing information from the encoder’ is to average

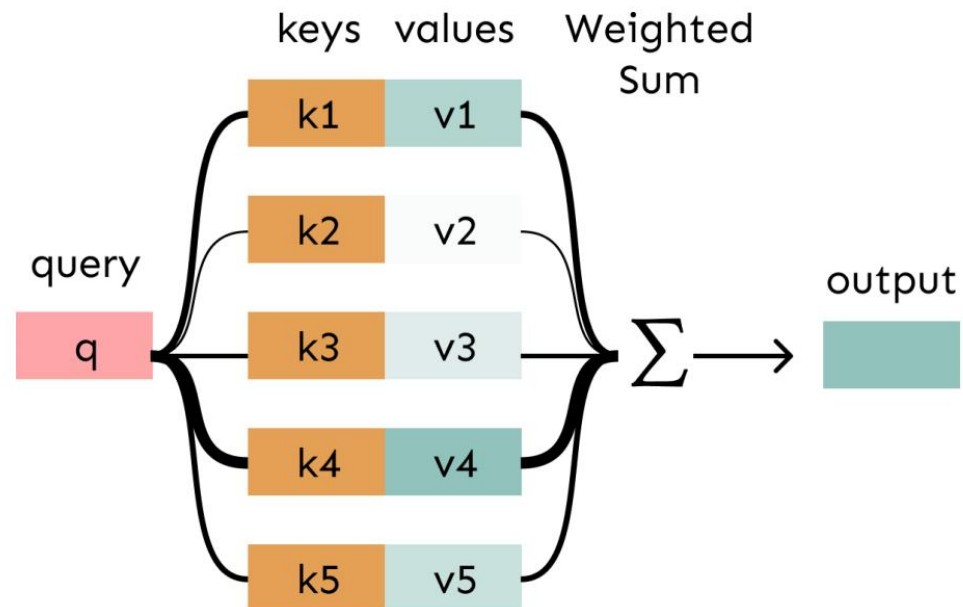


Attention

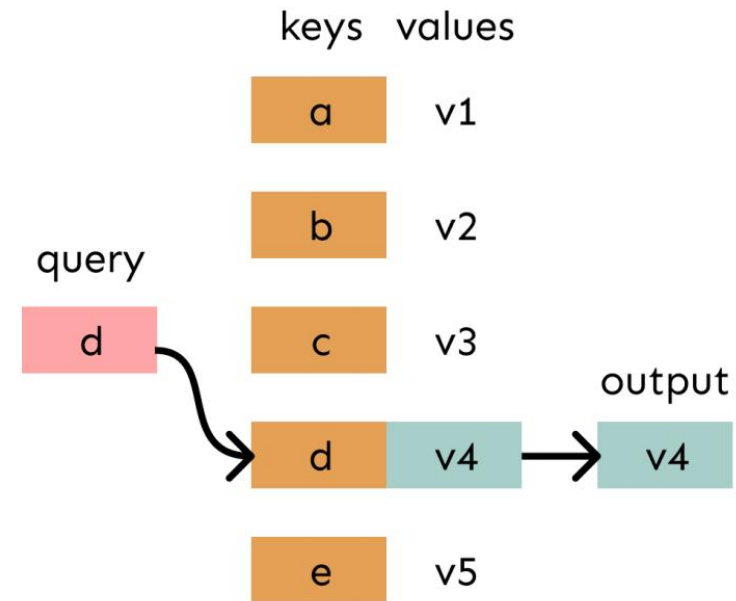


- Attention is **weighted averaging**, which lets you do lookups! This is very powerful if the weights are **learned**!

In **attention**, the **query** matches all **keys** *softly*, to a weight between 0 and 1. The keys' **values** are multiplied by the weights and summed.



In a **lookup table**, we have a table of **keys** that map to **values**. The **query** matches one of the keys, returning its value.



Attention



Think of attention as a soft dictionary lookup.

keys	food				
values	nutritive value	3	2	1.5	1

hard lookup  = 3 = $[1 \ 0 \ 0 \ 0] \cdot \begin{bmatrix} 3 \\ 2 \\ 1.5 \\ 1 \end{bmatrix}$

 = null

soft lookup  = $[0.3 \ 0.1 \ 0.6 \ 0] \cdot \begin{bmatrix} 3 \\ 2 \\ 1.5 \\ 1 \end{bmatrix} = 2$

attention scores

0.3 0.1 0.6 0

key points:

- input: query(Q)、key(K)、value(V)
- compute attention scores
- output: weighted value

example source: <https://zhuanlan.zhihu.com/p/143807314>

Attention: Q, K, V



	Represents	Its Role in Computation
Query(Q)	What the current token is looking for / interested in.	Used to compute similarity scores against all Keys.
Key(K)	What each token can provide or its identity for matching .	Matched against the Query to produce attention weights.
Value(V)	The actual information content of each token.	The vectors that are weighted and summed to produce the final output.

In a sequence to sequence model:

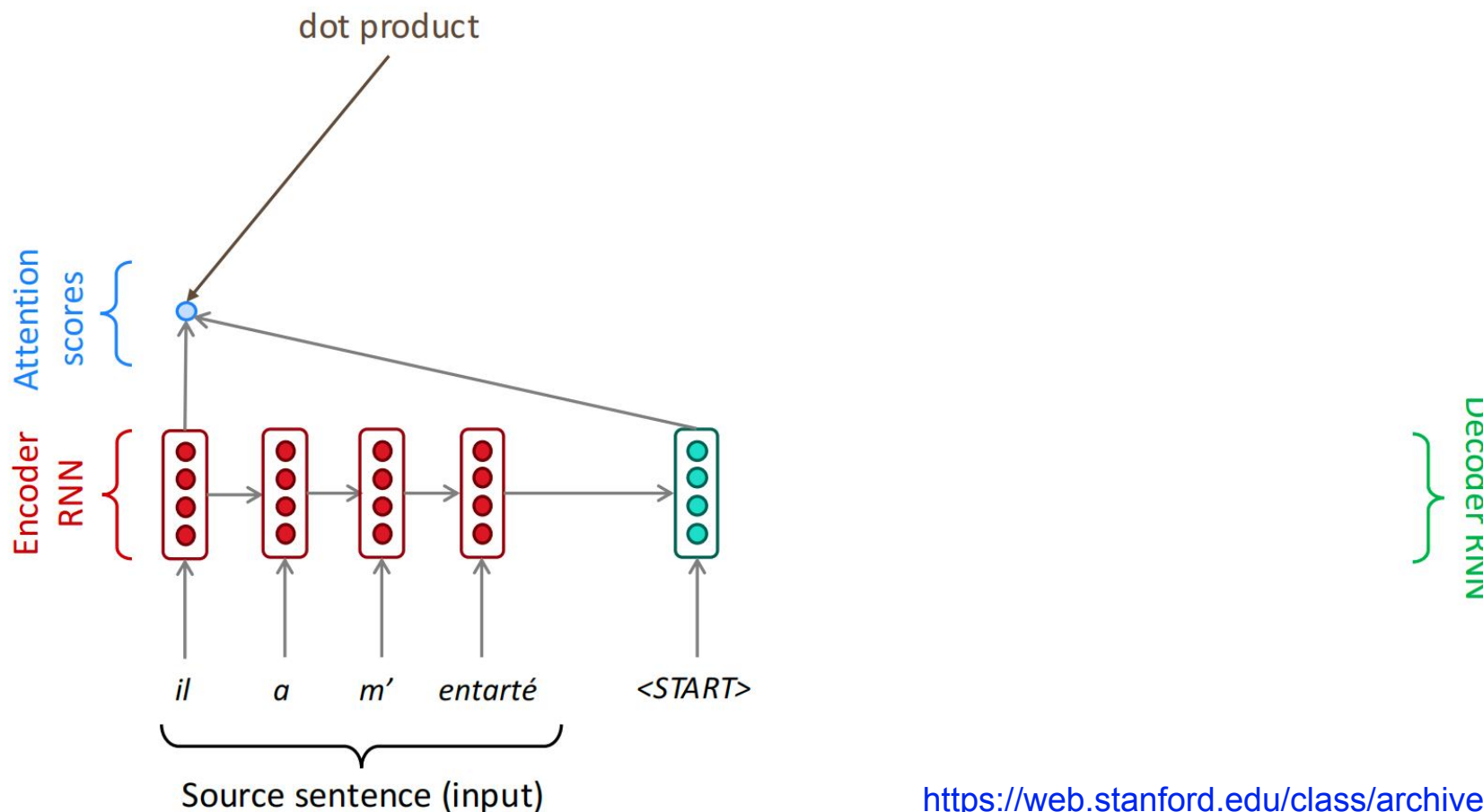
Q: the current hidden state in decoder s_t

K/V: the output of the encoder at each time step $h_1, \dots, h_N$

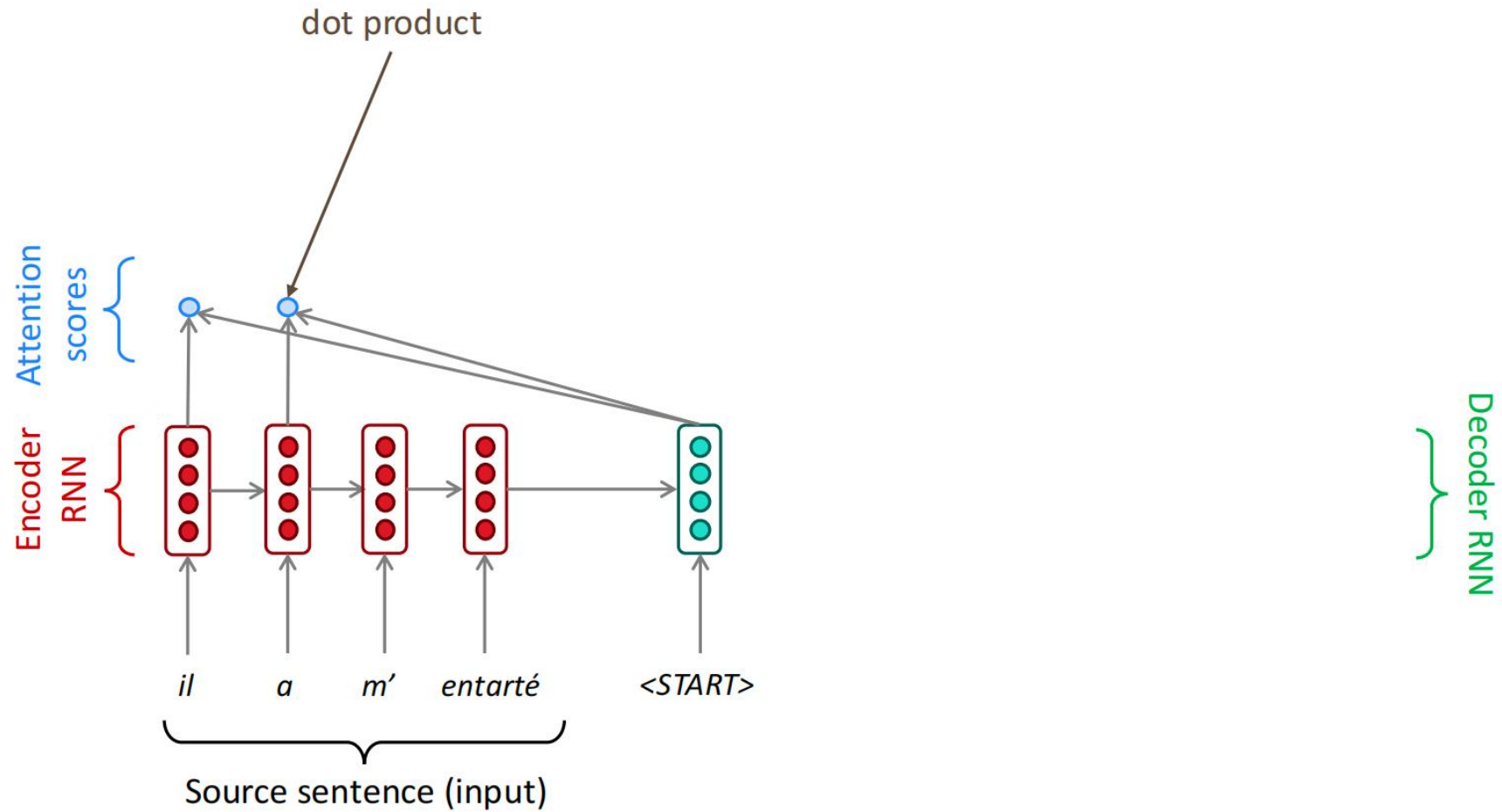
Sequence-to-sequence with attention



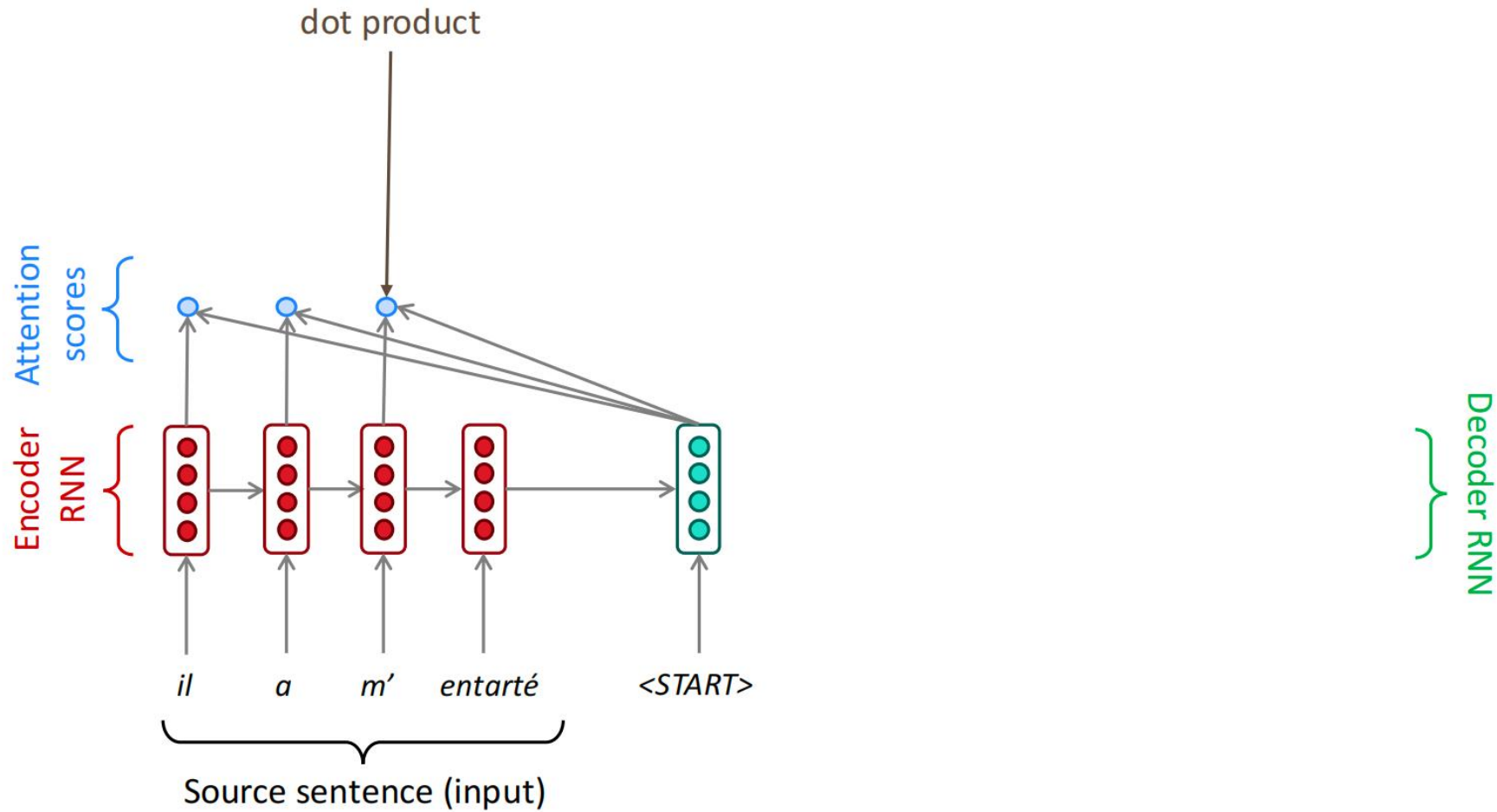
Core idea: on each step of the decoder, use direct connection to the encoder to focus on a particular part of the source sequence.



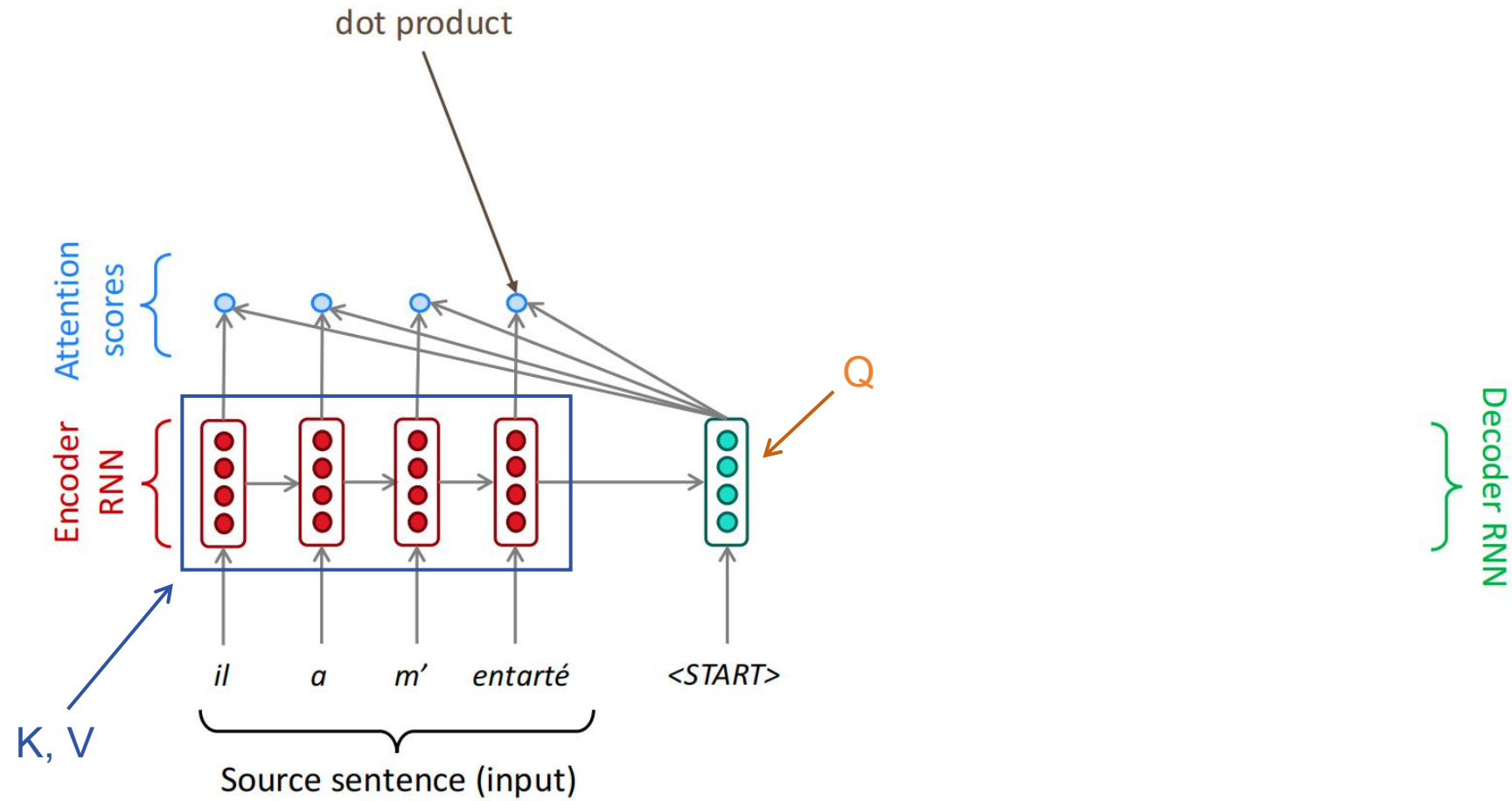
Sequence-to-sequence with attention



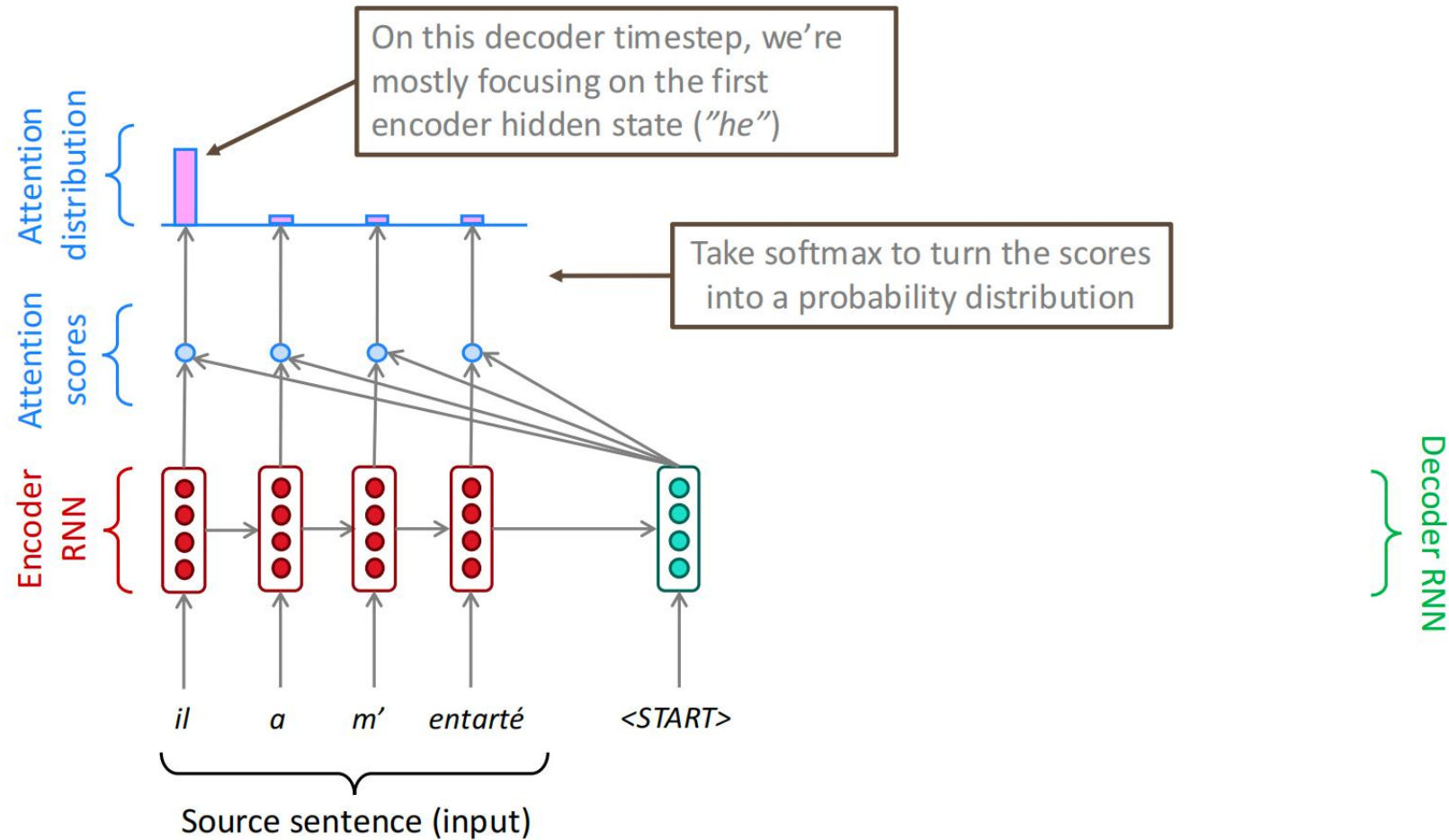
Sequence-to-sequence with attention



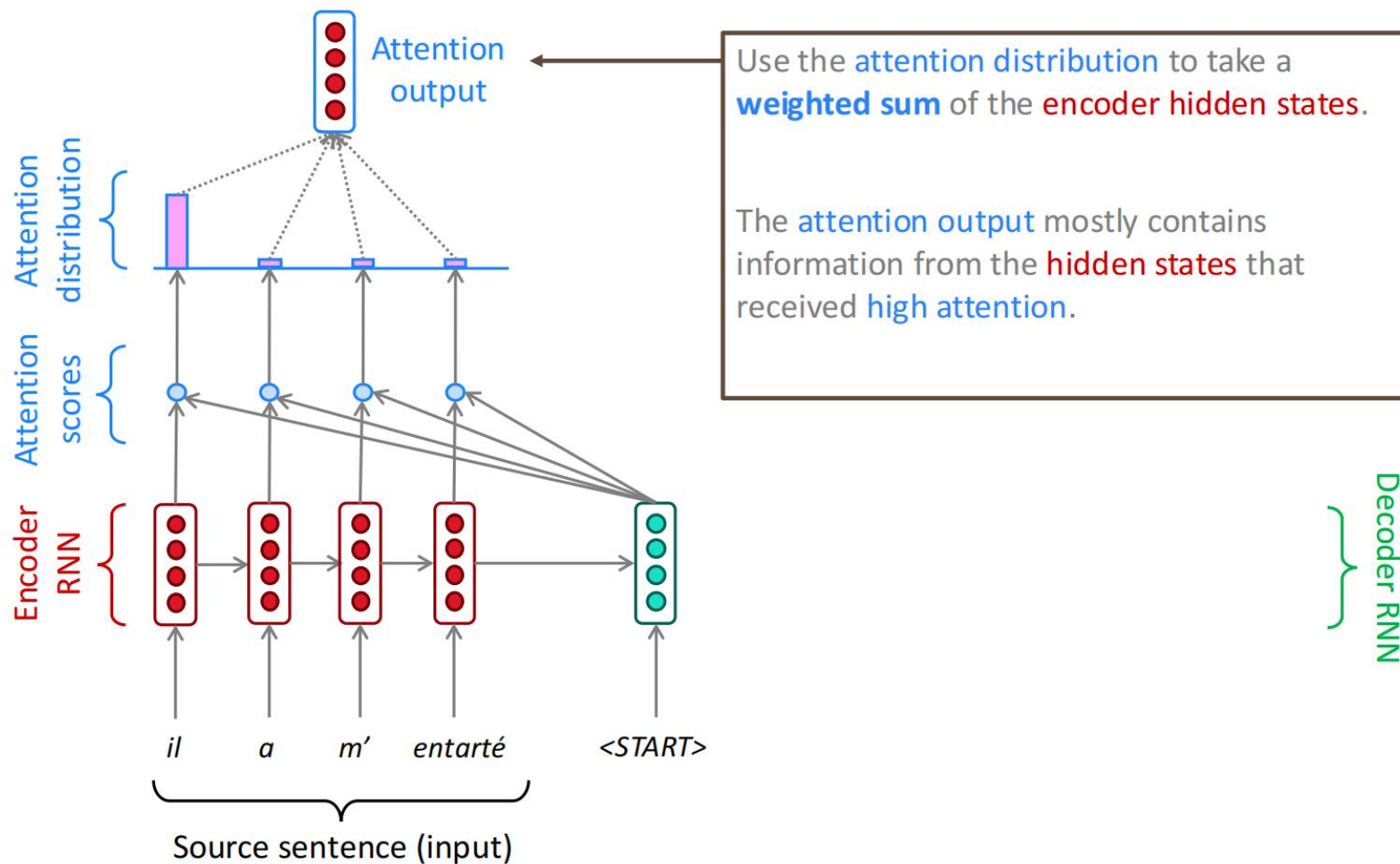
Sequence-to-sequence with attention



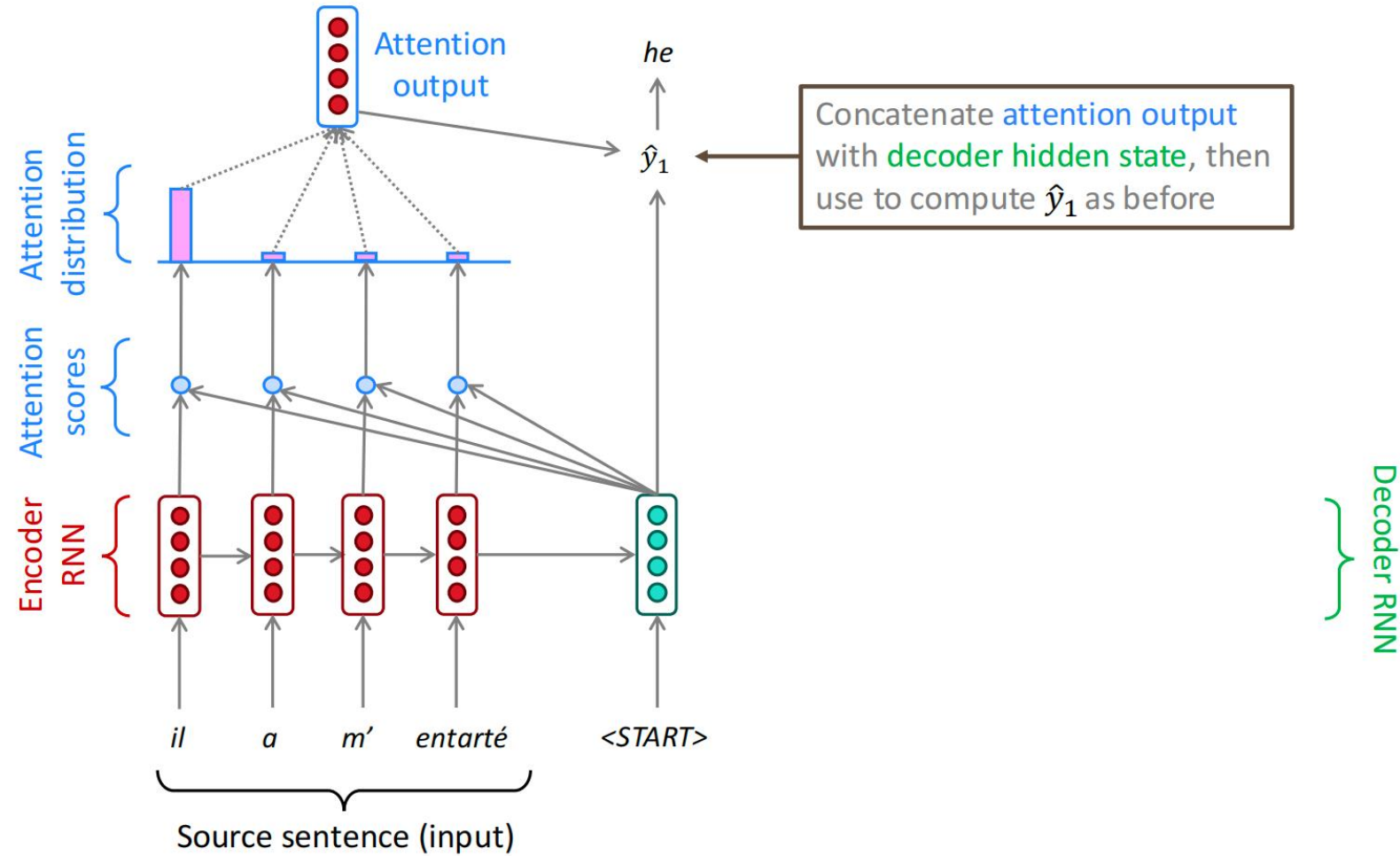
Sequence-to-sequence with attention



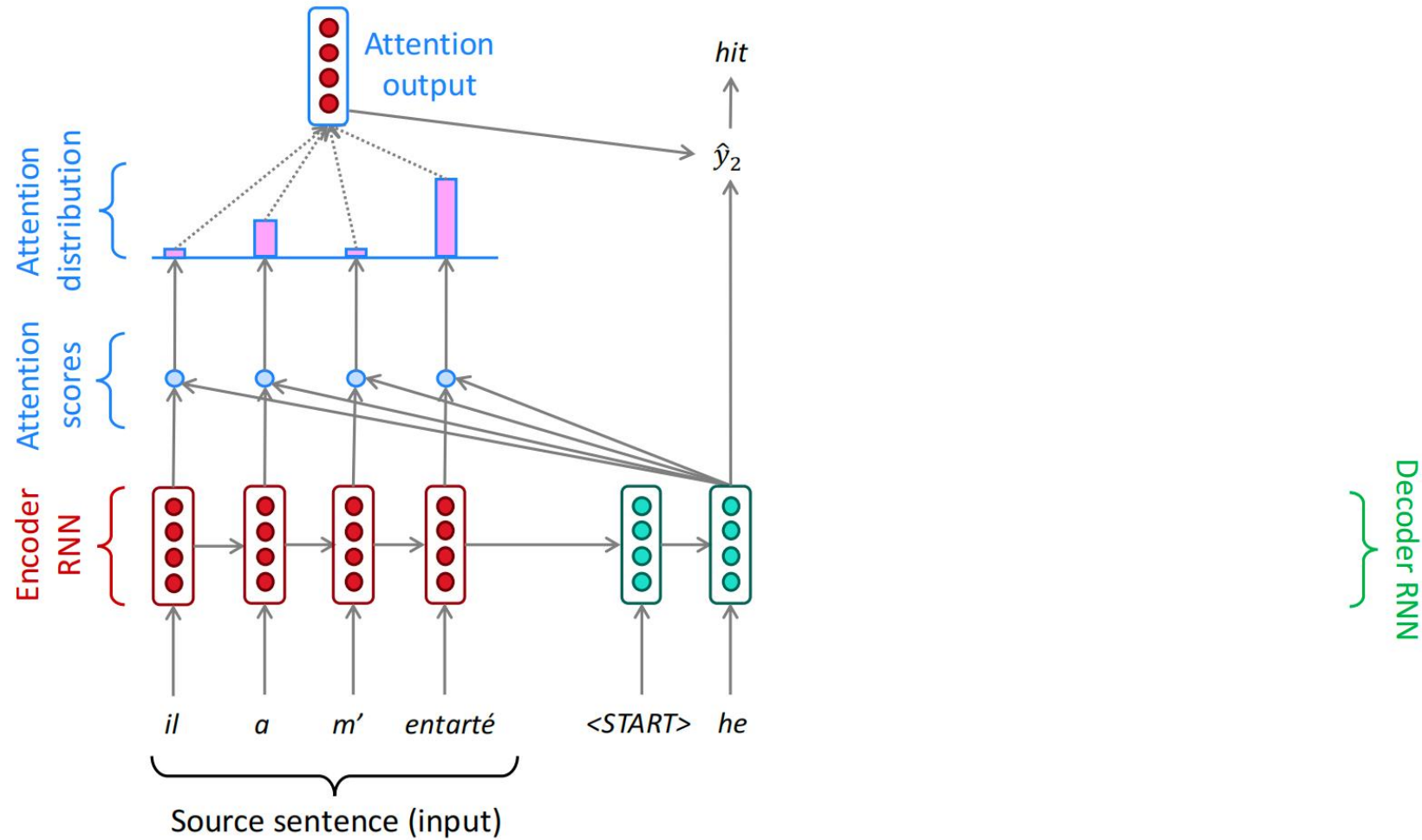
Sequence-to-sequence with attention



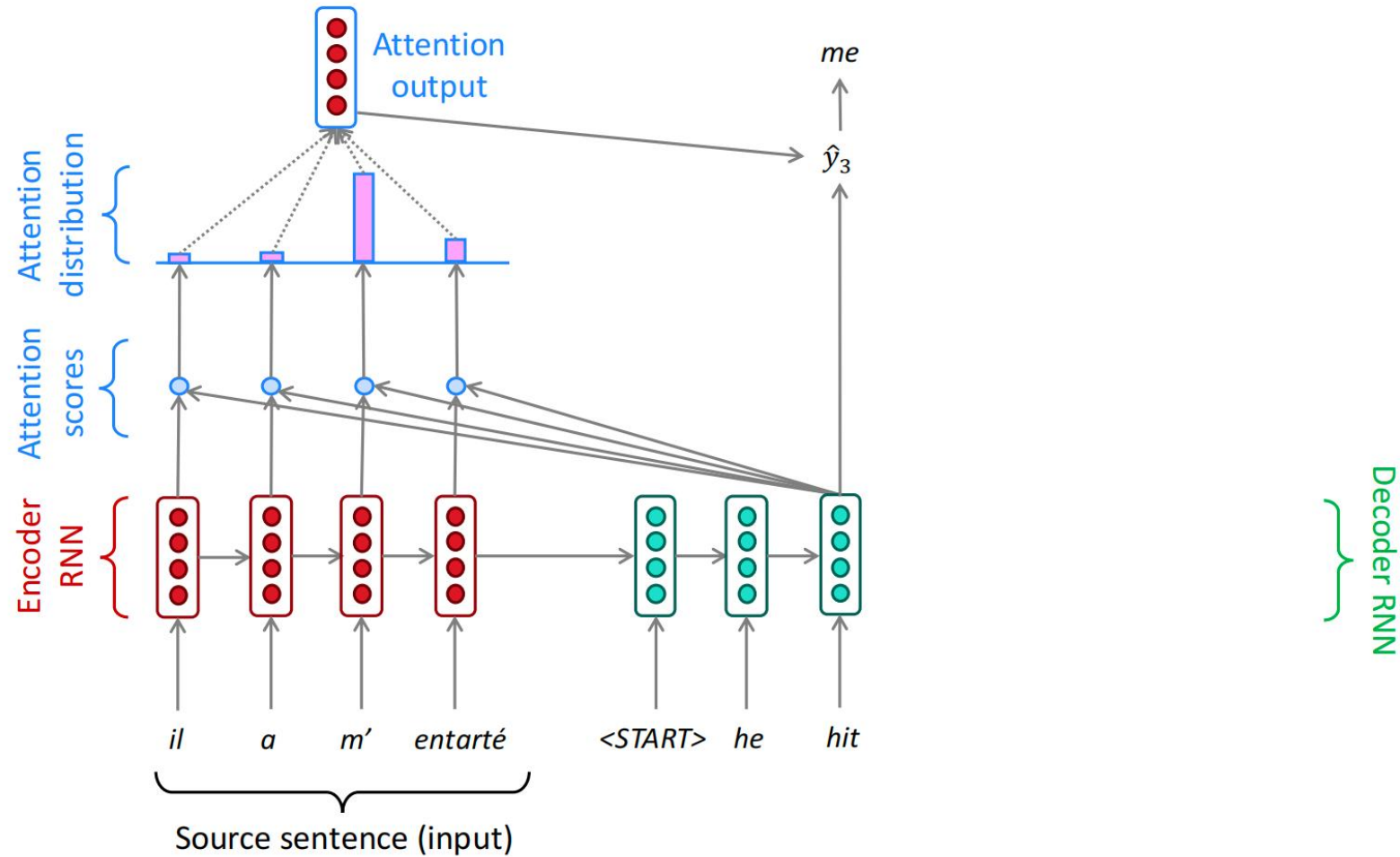
Sequence-to-sequence with attention



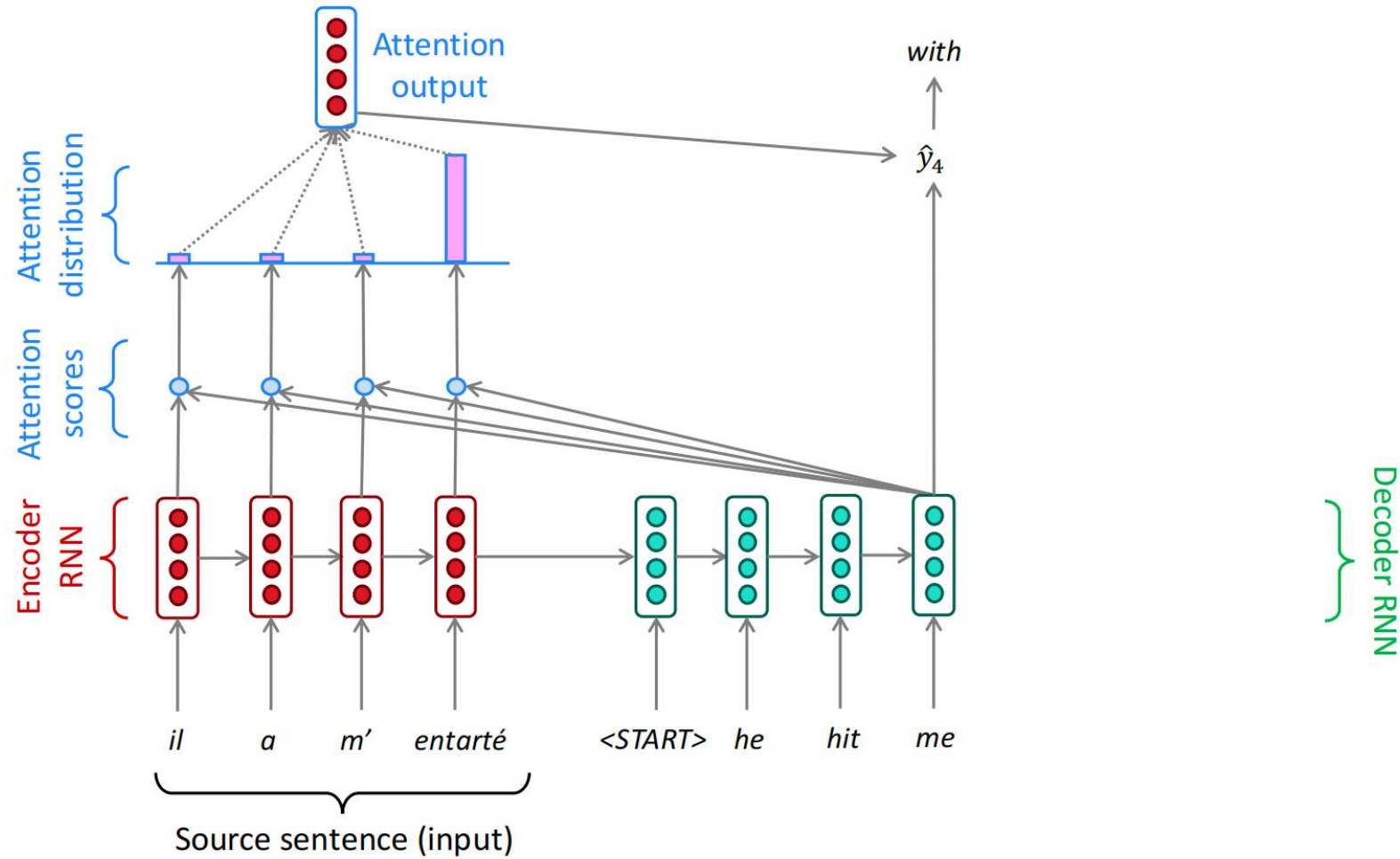
Sequence-to-sequence with attention



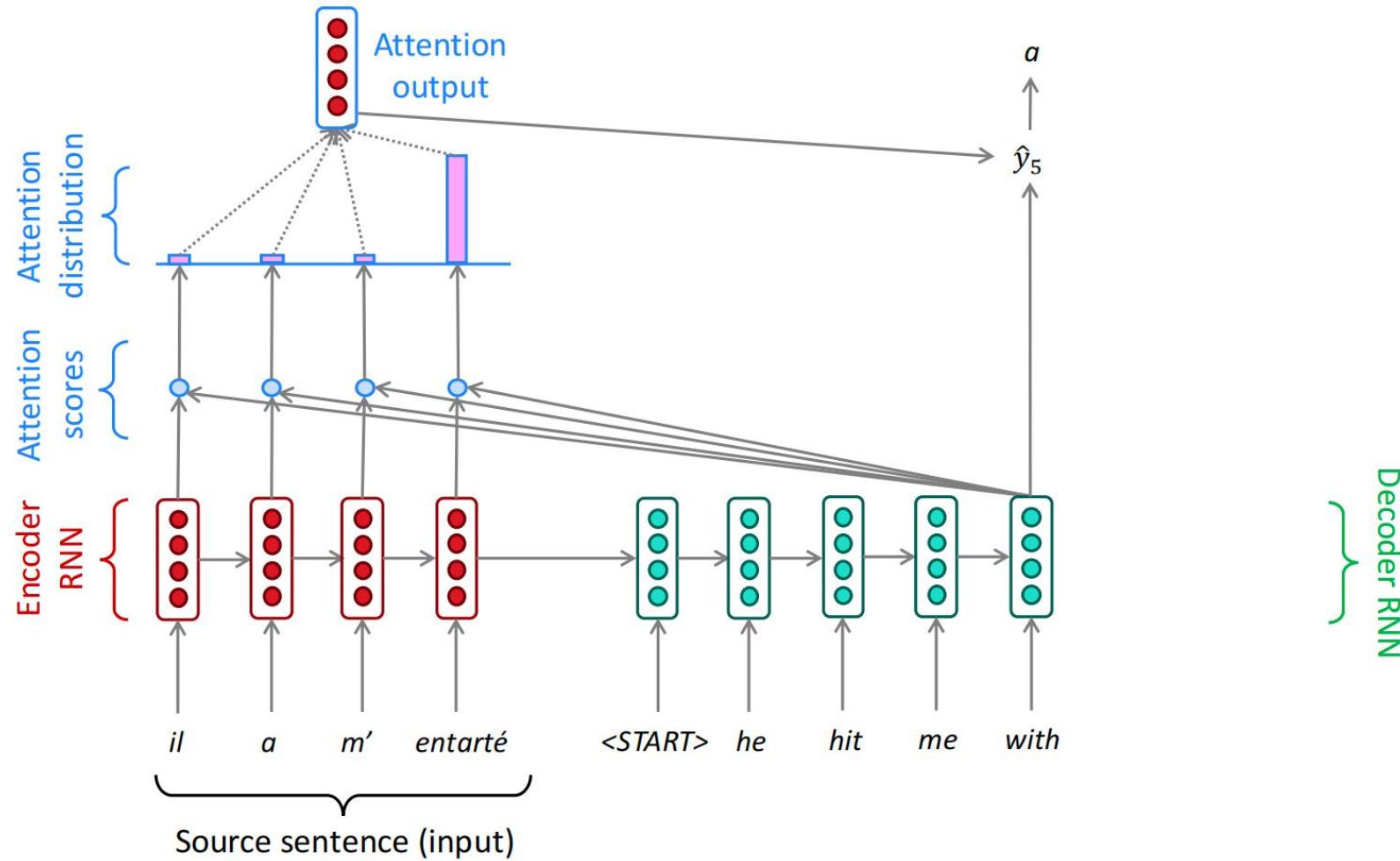
Sequence-to-sequence with attention



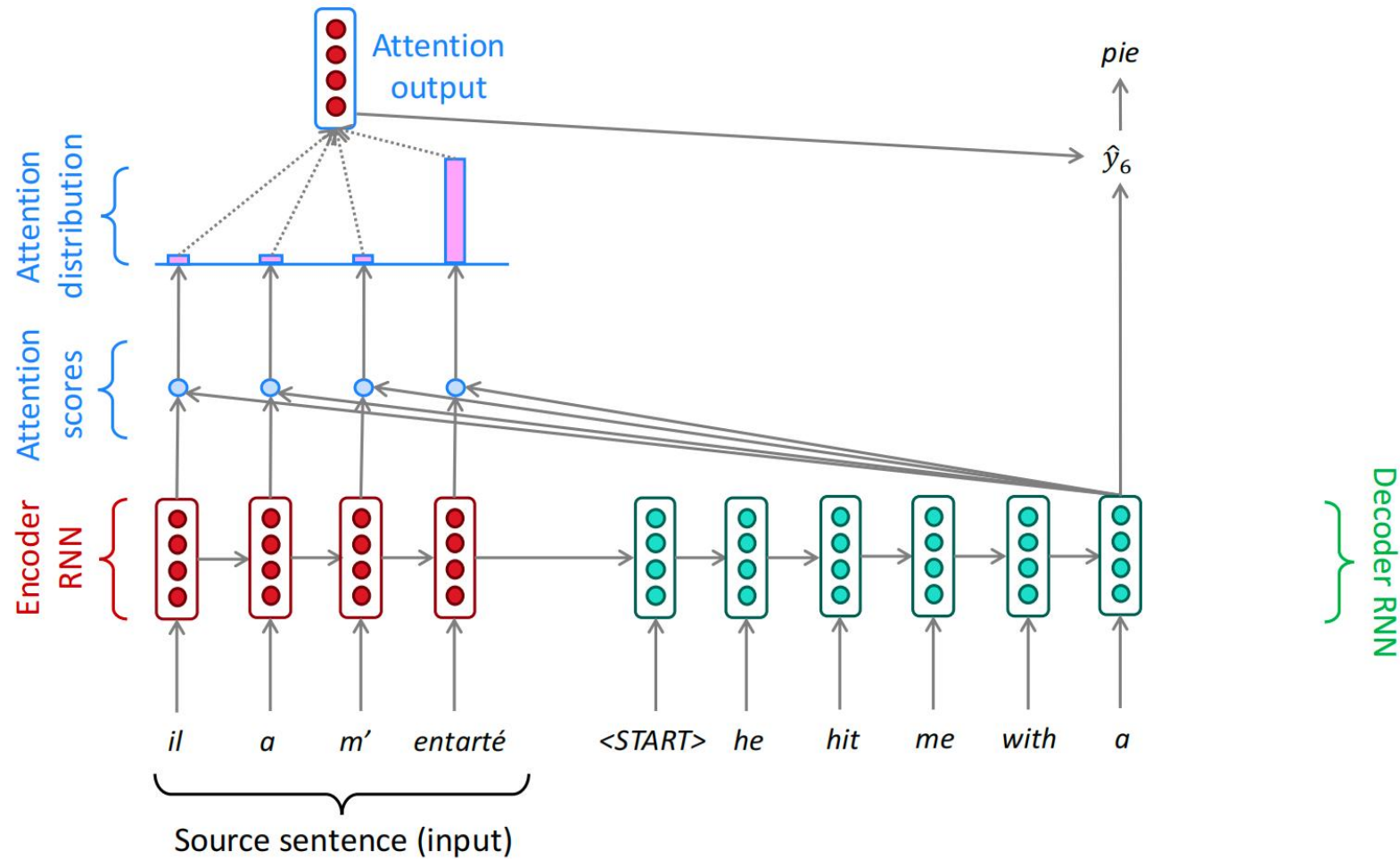
Sequence-to-sequence with attention



Sequence-to-sequence with attention



Sequence-to-sequence with attention



Sequence-to-sequence with attention



Equations:

- We have encoder hidden states: $h_1, \dots, h_N \in R^h$
- The current decoder hidden state is computed as $s_t = \text{RNN}_{\text{decoder}}(s_{t-1}, y_{t-1})$
- We get the attention scores for this step: $e_t = [s_t^T h_1, \dots, s_t^T h_N] \in R^N$
- We take softmax to get the attention distribution for this step (this is a probability distribution and sums to 1): $\alpha_t = \text{softmax}(e_t) \in R^N$
- We use α_t to take a weighted sum of the encoder hidden states to get the attention output(sometimes called the context vector) c_t : $c_t = \sum_i^N \alpha_{t,i} h_i \in R^h$
- An attentional hidden state is computed based on a weighted concatenation of the attention output and the current decoder hidden state: $\tilde{s}_t = \tanh(W_c [c_t; s_t])$
- The decoder output: $y_t = \text{softmax}(W_y \tilde{s}_t)$

Luong et al, 2015. Effective Approaches to Attention-based Neural Machine Translation. <https://arxiv.org/abs/1508.04025>

Attention is great!



■ Attention solves the bottleneck problem

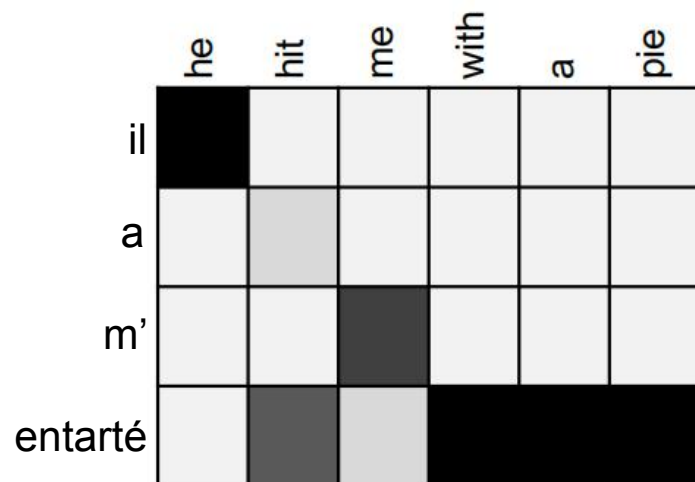
- Attention allows decoder to look directly at source; bypass bottleneck

■ Attention helps with the vanishing gradient problem

- Provides shortcut to faraway states

■ Attention provides some interpretability

- By inspecting attention distribution, we see what the decoder was focusing on



Attention: General Formula



Input

- Q: $\mathbf{s} \in R^{d_s}$
- K/V: $\mathbf{h}_1, \dots, \mathbf{h}_N \in R^{d_h}$

There are multiple ways to do this.

Compute attention scores

- Attention scores: $e_i = \text{score}(\mathbf{s}, \mathbf{h}_i) \in R$, $\mathbf{e} = [e_1, \dots, e_N] \in R^N$
- Attention distribution/weights: $\boldsymbol{\alpha} = \text{softmax}(\mathbf{e}) \in R^N$

Output

- Attention output: $\mathbf{c}_t = \sum_i^N \alpha_i \mathbf{h}_i \in R^{d_h}$



Attention Variants

There are several ways you can compute $\mathbf{e} \in R^N$ from $\mathbf{h}_1, \dots, \mathbf{h}_N \in R^{d_h}$ and $\mathbf{s} \in R^{d_s}$:

- Basic dot-product attention: $e_i = \mathbf{s}^T \mathbf{h}_i \in R$
 - This assumes $d_h = d_s$
- Multiplicative attention: $e_i = \mathbf{s}^T \mathbf{W} \mathbf{h}_i \in R$
 - Where $\mathbf{W}_{d_s \times d_h}$ is a weight matrix
- Reduced-rank multiplicative attention: $e_i = \mathbf{s}^T (\mathbf{U}^T \mathbf{V}) \mathbf{h}_i = (\mathbf{U} \mathbf{s})^T (\mathbf{V} \mathbf{h}_i) \in R$
 - For low rank matrices $\mathbf{U} \in R^{k \times d_s}$, $\mathbf{V} \in R^{k \times d_h}$, $k \ll d_s, d_h$
- Additive attention: $e_i = \mathbf{v}^T \tanh(\mathbf{W}_1 \mathbf{h}_i + \mathbf{W}_2 \mathbf{s}) \in R$
 - Where $\mathbf{W}_1 \in R^{d \times d_h}$, $\mathbf{W}_2 \in R^{d \times d_s}$ are weight matrices and $\mathbf{v} \in R^d$ is a weight vector.
 - d (the attention dimensionality) is a hyperparameter

Attention in
Transformer

Attention



- Attention is a general Deep Learning technique.
 - We've seen that attention is a great way to improve the sequence-to-sequence model for Machine Translation.
 - However: You can use attention in **many architectures** (not just seq2seq) and **many tasks** (not just MT)